

AGENTKERNEL

The Trust-Native Agentic Operating System

A Paper from the DeepKernel Lab

Zhenhua Zou Sheng Guo Qiuyang Zhan

Lepeng Zhao Shuo Li Zhuotao Liu[†]

[†]Corresponding author: zhuotaoliu@tsinghua.edu.cn

Abstract

Modern AI agents routinely cross trust boundaries especially in long-horizon tasks: they ingest untrusted web and repository content, fuse it with privileged system instructions, persist intermediate beliefs in long-term or persistent memory, and invoke privileged tools. This pipeline creates a broad surface for malicious payloads to infiltrate through the model’s inputs and execute harmful actions via tool calls, raising serious concerns about the level of autonomy we should grant AI agents and also driving a surge in commercial AI agent safeguarding tools. However, even today’s most capable governance stacks remain application-level middleware; they co-reside with agents inside a shared process trust boundary. We argue that what is missing is not another security policy or plugin, but an *operating-system substrate* that supplies mandatory, non-bypassable services for identity, input mediation, memory governance, and execution control.

To systematically close the trust gap, we introduce AGENTKERNEL, the first *trust-native agent operating system* built around a single premise: security is the first-class design constraint, not an afterthought bolted onto an LLM runtime. AGENTKERNEL wraps the agent lifecycle in a mandatory enforcement boundary organized into four pillars—Identity, Perception, Cognition, and Execution—each grounded in classical OS security ideas and lifted to the semantic plane where autonomous agents actually fail (delegation, prompt injection, memory poisoning, and tool misuse).

Our central claim is that *structural security is a capability multiplier*. Kernel-managed identity enables trustworthy cross-organization collaboration; graduated perception replaces brittle single-point filters; information-flow-controlled memory improves retrieval fidelity while blocking poisoning; and semantic-to-kernel execution enforcement lets operators grant broader tool privileges because the boundary is architecturally non-bypassable. We position AGENTKERNEL as the missing OS layer beneath the growing harness ecosystem—orchestration frameworks, agent runtimes, governance platforms, and execution sandboxes—and argue through systematic comparison that AGENTKERNEL is the first integrated agent OS in which security is the organizing principle across the full lifecycle, enabling stronger agents rather than merely constraining them.

1 Introduction

LLM-based AI agents are ceasing to be chat demos and are becoming the *control plane* through which organizations read, edit, and operate software: a single stack that couples an LLM to private repositories, terminals, issue trackers, and—increasingly—other agents. The first wave that made this legible to practitioners is commercial *coding-agent* products and IDE-native assistants (e.g., Cursor [1], Claude Code [2], and GitHub Copilot/OpenAI Codex-class systems [3, 4]) that already sit inside everyday engineering workflows. Parallel to those products, high-visibility open-source harnesses such as OpenClaw [5, 6] and Nous Research’s Hermes agent [7] have extended the boundary of general AI agents by incorporating diverse message channels for better accessibility and using self-evolving skills for better personalization. The research lineage

for tool use, delegation, and long-horizon workflows is well established [8, 9, 10], and the library layer has scaled accordingly: representative frameworks including AutoGPT [11], LangChain [12], OpenHands [13], and MetaGPT [14] now count hundreds of thousands of GitHub stars in aggregate, while enterprise-backed stacks such as Microsoft’s Agent Framework [15], CrewAI [16], and Semantic Kernel [17] underpin production orchestration. Mobile and desktop “computer use” copilots extend the same pattern to full GUI surfaces [18, 19]. Across products, viral harnesses, and research prototypes, the architectural constant is unchanged: an LLM fused with multi-source context and high-privilege tools, without an operating system co-designed to mediate that fusion end-to-end.

Traditional applications were born into an environment already rich with OS-level mediation infrastructure: process isolation, virtual memory, file systems, inter-process communication, and mandatory access control. These services are so fundamental that no one would attempt to build a production application without them. AI agents, by contrast, operate in an infrastructural vacuum. They share identity namespaces (self-declared strings), receive unmediated perception (no reliable mediation between untrusted content and the LLM context), lack provenance-aware memory management (no mandatory taint or information-flow discipline), and execute actions with vague permissions or unconstrained privileges. The upshot is not merely a security gap but an *infrastructure gap*: the same missing substrate that blocks trustworthy deployment also caps reliability, auditability, and cross-agent collaboration at scale.

The ecosystem has nevertheless filled up with partial remedies. We catalogue them—as we later formalize in §6—into four harness tiers that loosely track the lifecycle dimensions of §2 (identity, perception, cognition, execution). *Orchestration frameworks* such as LangChain/LangGraph [12, 20], AutoGen [21], CrewAI [16], Microsoft’s Agent Framework [15], and Semantic Kernel [17] optimize composable workflows and tool routing. *Agent runtimes* including AIOS [22], OpenFang [23], SmythOS SRE [24], and Letta [25] combine scheduling and observability with layered memory (Letta) and, for some stacks, WASM- or cloud-backed resource isolation (OpenFang, SmythOS). *Governance platforms*—for example Microsoft’s Agent Governance Toolkit (AGT) [26] alongside tracing stacks such as LangSmith [27] and AgentOps [28]—supply policies, approvals, and audit pipelines for production agents. *Execution sandboxes* such as nono [29], E2B [30], and Anthropic’s sandbox-runtime [31] isolate OS- or cloud-level compute from the host. Each tier advances an important slice of the stack, yet none is specified as a single mandatory kernel that mediates every crossing between untrusted input, protected cognition, and auditable output.

Against classical OS guarantees, today’s harness exhibits three recurring structural weaknesses. *Fragmentation*: identity signals, perception filters, memory stores, tool executors, and policy engines ship as loosely coupled packages with incompatible provenance labels and no uniform IPC substrate, so compromising or bypassing one component often voids assumptions elsewhere. *Blurry trust boundaries*: even flagship governance stacks remain co-located with agent business logic—AGT deliberately offers “application-level governance, not OS kernel-level isolation,” with agents and its policy engine sharing a single process trust boundary [26], so policy, prompts, and tools interleave and neither security nor application surfaces remain independently auditable or replaceable. *Non-mandatory enforcement*: defenses appear as optional hooks or middleware that may inspect tool *parameters* yet cannot bind the *actual* syscall layout—a child process spawned inside a tool can still escalate—while sandboxes confine execution without governing how untrusted perception poisons long-lived memory.

A motivating scenario. Consider a multi-agent DevOps pipeline: an orchestrator delegates pull request review and conditional deploy to specialist agents. A crafted PR comment carries an indirect prompt-injection payload. Without cryptographically grounded delegation, the reviewer cannot authenticate who authorized the task; without kernel-mediated perception, the payload reaches the model; without taint-aware

memory, the poisoned summary is archived and later retrieved as trusted context; without semantic-to-syscall enforcement, a deployer acts on that summary and pushes malicious code. Each hop is independently realistic [32, 33, 6]. Abstracting from the story, the failure is less a patchwork of unrelated bugs than the absence of a *composable* mediation narrative: a traditional OS prevents analogous application compromises because isolation, mandatory access control, and provenance are always on, span every resource transition, and compose under one reference monitor. Agent workloads need the same guarantee lifted to the *semantic* plane—prompt injections instead of buffer overflows, conversational context instead of raw address spaces—while still anchoring to the real syscall surface if enforcement is to be non-bypassable. Point solutions reshuffle symptoms; they do not change the end-to-end trust calculus of a session. What is missing is therefore not another harness feature but an *agent operating system*: a mandatory layer that unifies identity, perception, cognition, and execution mediation.

Proposal and goals. We propose AGENTKERNEL, the first trust-native *agent operating system* whose purpose is to close the structural gap catalogued above: not by accumulating yet more ad hoc guardrails, but by supplying a *single, systematically specified, mandatory mediation layer* that is native to how LLM-based agents actually move trust—through delegation, context assembly, durable memory, and tool-mediated side effects. Our objective is practical as much as conceptual: make agent sessions *auditable*, *composable*, and *deployable at scale* under realistic environments (richer inputs, broader tools, cross-organization collaboration), so that stronger autonomy becomes a reliability and assurance story rather than a latent incident report.

Design at a glance. AGENTKERNEL wraps the agent core in a security kernel that every crossing must traverse. *Identity* elevates “who is acting” from self-declared strings to cryptographically grounded enrollment, capability chains, and mutual attestation so that orchestration, delegation, and agent-to-agent handoffs carry verifiable provenance. *Perception* replaces single brittle filters with a graduated pipeline that treats untrusted web, repository, and conversational content as labeled inputs whose risk is reduced *before* it becomes indistinguishable from system instructions in the model context. *Cognition* governs what may be remembered and retrieved: memory is not a passive vector database but an information-flow-disciplined store in which taint and policy lattice semantics propagate to individual items, so contaminated beliefs cannot silently masquerade as operational facts. *Execution* closes the last mile from declared tool intent to actual host behavior through a staged bridge that binds semantic permissions to syscall-level enforcement (e.g., eBPF hooks and process-tree monitoring), so that privilege escalation inside a tool implementation cannot outrun the kernel’s view of what was authorized.

Why this is one system, not a toolkit mash-up. The four pillars are not interchangeable plugins; they instantiate a *shared reference-monitor discipline* adapted to the semantic plane. Classical ideas—non-bypassable mediation, deny-by-default composition, defense-in-depth, minimal integration surface—are instantiated once, coherently: identity establishes the principal; perception assigns initial labels; cognition preserves and propagates those labels across time; execution enforces the resulting obligations on outward actions. Returning to the motivating DevOps pipeline (§1), the failure chain is not four unrelated bugs but four missing transitions in the *same* end-to-end story: impersonation without grounded identity, injection without kernel-mediated perception, poisoning without taint-aware cognition, and over-privileged side effects without semantic-to-syscall execution. AGENTKERNEL targets that narrative directly: each pillar supplies an independent invariant so that a compromise in one layer does not automatically collapse the others, while provenance and policy flow downward so the system remains a single auditable whole.

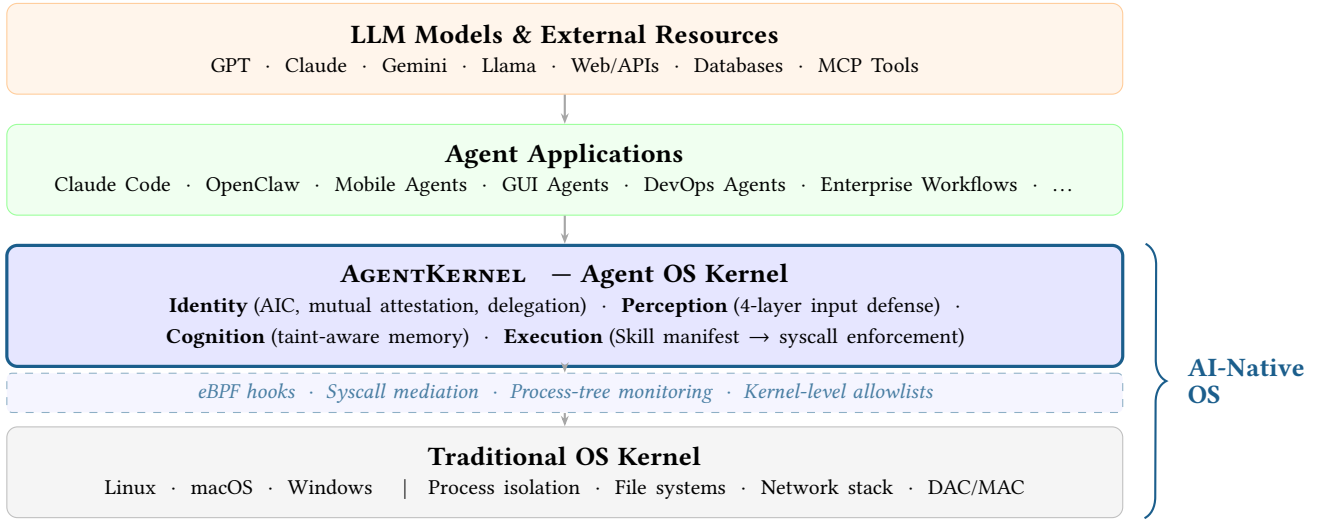


Figure 1: *AGENTKERNEL’s position in the AI agent technology stack. Agent applications (Claude Code, OpenClaw, mobile/GUI agents, etc.) sit atop AGENTKERNEL, which mediates all interactions with LLM models, tools, and the outside world. AGENTKERNEL and the traditional OS kernel jointly form the AI-Native OS: AGENTKERNEL provides semantic-level services (identity, perception, cognition, execution governance) while leveraging the traditional kernel for syscall-level enforcement via eBPF hooks and process-tree monitoring.*

Ecosystem positioning. Figure 1 situates AGENTKERNEL between agent applications—commercial coding agents, viral open harnesses, mobile and desktop “computer use” stacks—and the traditional OS kernel. AGENTKERNEL does not supplant Unix-like kernels; it complements them: the traditional kernel remains authoritative for address spaces, devices, and low-level isolation, while AGENTKERNEL becomes authoritative for *agent semantics*—who may speak for whom, which content may enter the LLM context, how memory items inherit trust, and which tool behaviors are actually permitted on the host. Together, the two layers form what we call an *AI-Native OS*: semantic mediation with a syscall backstop. Host kernels already enforce isolation and mandatory access over kernel-addressable objects—files, processes, sockets—yet the residual problem is not another layer of labels on those objects. Agents fuse untrusted natural language with privileged automation across sessions, organizations, and long horizons, so the crossings that must be mediated include intents, memories, and plans that kernels neither model nor observe. AGENTKERNEL supplies the missing mandatory services on the semantic plane while anchoring outward effects to the traditional kernel where they ultimately become syscalls. We borrow from the MAC era only its *architectural shape*—always-on, non-bypassable mediation that composes into a deployable trust story—rather than duplicating kernel object enforcement. Treating structural security this way makes it a *capability multiplier*: operators can justify richer untrusted inputs, broader tool privileges, and cross-domain collaboration because the boundary is non-bypassable by construction. The remainder of this section summarizes our contributions.

Contributions. This paper contributes the following, in service of the positioning above:

1. A **unified lifecycle diagnosis** that reframes today’s agent stacks around four trust-critical phases—identity, perception, cognition, and execution—and analyzes what shape of OS-level services the model AI agents need and why traditional low-level OS primitives fall short. We go deeper to reveal the security and capabilities consequences of the lack of such semantic-layer OS mediations, motivating our insight that an OS-shaped substrate is prerequisite rather than optional (§2).
2. The **AGENTKERNEL architecture and design principles**: a mandatory security kernel organized into the four pillars above, guided by five principles distilled from classical OS security (external reference monitor, deny-by-default policy intersection, lifecycle defense-in-depth, semantic mediation with syscall

backstop, and a narrow agent–kernel integration boundary to model, tool, and storage capability). We present the two trust anchors (a remote registry for enrollment and credentials, and a local agent kernel), the three integration adapters through which frameworks attach, and the end-to-end data and control paths that keep mediation outside the manipulable LLM context (§3, §4).

3. A **structured security analysis** that states explicit properties per pillar, articulates how cryptographic, information-flow, and kernel-backed guarantees compose, and clarifies the residual threat model—including what remains when policies are mis-set and how the semantic–syscall bridge is intended to behave under stress (§5).
4. A **scientific positioning** of AGENTKERNEL within the rapidly evolving harness landscape: a tiered map of orchestration frameworks, agent runtimes, governance platforms, and execution sandboxes, followed by a systematic comparison to representative systems (e.g., Microsoft’s Agent Governance Toolkit, AIOS-class runtimes, and sandbox offerings). The goal is not to claim feature parity but to show where mandatory lifecycle mediation is absent today and how an OS-first layer complements capability-first and policy-first stacks (§6).

Roadmap. §2 develops the lifecycle model and threat surface. §3 and §4 present the design principles and architectural instantiation of AGENTKERNEL. §5 analyzes security properties and composition. §6 maps the broader ecosystem, compares against related systems, and argues for the OS layer’s role in the trusted computing base. §7 discusses limitations and research directions, and §8 concludes.

2 Background and Motivation

2.1 The Agent Lifecycle

A modern LLM-powered agent operates through a continuous loop of *perception*, *cognition*, and *execution*. During **perception**, the agent ingests inputs from diverse sources—user messages, tool outputs, web pages, API responses, and messages from other agents. During **cognition**, it reasons over these inputs, retrieves and updates long-term memory, formulates plans, and decomposes goals into sub-tasks. During **execution**, it invokes tools, calls APIs, writes files, sends messages, or delegates sub-tasks to other agents. Cutting across all three phases is **identity**: the agent must establish *who* it is, *what* it is permitted to do, and *whom* it is interacting with.

We conceptualize the agent’s operating environment as three trust domains, illustrated in Figure 2:

- **Input World** (untrusted by default): user instructions, external agents, tool outputs, web content, system events.
- **Agent Core** (protected): reasoning engine, context and memory stores, session and trajectory state.
- **Output World** (auditable, least-privilege): tool execution, data sinks, agent-to-agent delegation, audit logs.

Every transition between domains represents a *trust boundary crossing* that requires mediation. Yet in most contemporary agent frameworks, these crossings are unguarded.

2.2 What Agents Need from an Operating System, and Why Classical Primitives Fall Short

Traditional applications benefit from four categories of OS services: *identity and access control* (UIDs, file permissions, SELinux labels), *input mediation* (device drivers, protocol parsers), *memory management* (virtual memory, page tables, copy-on-write), and *execution governance* (process isolation, capabilities, sandboxing). By analogy, AI agents require semantic-level counterparts to these services:

1. **Identity management.** Agents need cryptographic identity that binds who built the agent, what code it runs, and who authorized its deployment—enabling trustworthy A2A collaboration and accountable

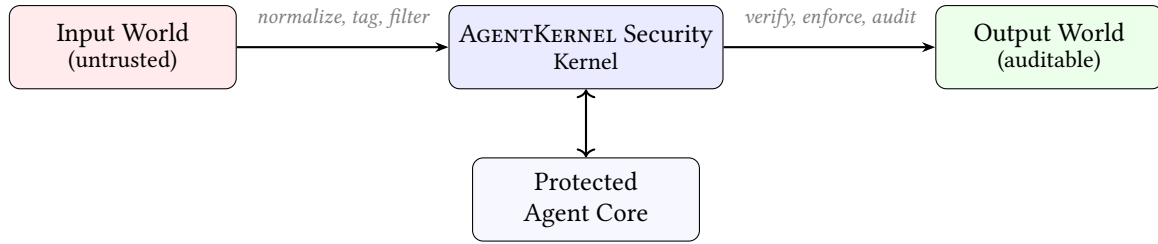


Figure 2: Three-domain model. All inputs traverse the AGENTKERNEL kernel before reaching the protected agent core; all outputs are identity-verified, intent-aligned, and permission-converged before leaving the kernel.

delegation chains.

2. **Input mediation.** Content from untrusted sources must be normalized, tagged, and filtered *before* entering the LLM context—not as an optional guardrail, but as a mandatory OS service analogous to a device driver sanitizing hardware input (cf. Figure 2).
3. **Memory management with provenance.** As agents gain persistent memory, they need information-flow control that tracks the origin and trustworthiness of each memory item—enabling better retrieval decisions and preventing persistent contamination.
4. **Execution governance.** Agent actions must be mediated from semantic intent down to actual system calls, ensuring that declared permissions match runtime behavior—enabling operators to safely grant broader tool access with confidence that the enforcement boundary holds.

Interpreting the lifecycle of §2.1, these four counterpart services respectively target *Identity*, inputs at the *Perception* boundary, provenance-aware *Cognition* state, and policy-convergent *Execution*—the same dimensions used in Table 1—and correspond to the mediated trust-boundary crossings in Figure 2.

Traditional operating systems nevertheless provide powerful primitives at a lower layer of abstraction: discretionary and mandatory access control, process isolation, sandboxing, and network filtering. These mechanisms operate at the *syscall level*—they mediate access to files, sockets, and devices. However, agent-specific gaps operate at the *semantic level*:

- An agent may present valid credentials, file descriptors, or IPC handles while syscall policies remain blind to cryptographic binding between running code, its builder, and the authorizing principal for a delegation chain.
- A prompt injection embedded in a tool’s JSON response is well-formed data—it violates no system call policy.
- An agent “legitimately” calling a file-write API but with content derived from a poisoned memory entry is invisible to syscall-level checks.
- An agent delegating to a sub-agent with escalated capabilities is a normal IPC operation from the OS perspective.

The gap between OS-level services and agent-level services is analogous to the gap that motivated application-layer firewalls: just as TCP/IP packet filtering cannot detect SQL injection, syscall mediation cannot detect prompt injection (we use this comparison only as intuition, not as a formal equivalence claim). What is needed is a new layer of *semantic-level OS services* that understands agent intents, memory provenance, and capability boundaries—while still leveraging OS-level enforcement as the ultimate backstop.

2.3 Infrastructure Gaps and Their Consequences

Absent or incomplete semantic-layer services couple security breaches with artificial capability ceilings. Table 1 summarizes representative paired consequences along each lifecycle dimension.

Table 1: *Infrastructure gaps across the agent lifecycle: missing OS services create both security vulnerabilities and capability limitations.*

Dimension	Security Consequence	Capability Consequence
Identity	Impersonation and unauthorized multi-hop delegation without binding principals to code and authorization Unverifiable third-party skills/plugins enabling supply-chain capability injection	Verifiable delegation chains across organizations stall; operators fall back on pairwise trust or manual approval gates Version freezes and strict whitelists block safe participation in open third-party capability ecosystems
Perception	Prompt injection via tools, web pages, and files; untrusted content co-mingled with privileged instructions Encoding bypasses, jailbreaks, and spoofed UI or sensor readings that misstate device or application state	Browser- and repository-backed toolchains are avoided or over-stripped, narrowing tasks an agent can safely cover Multimodal and high-entropy pipelines require human checkpoints, capping throughput and end-to-end autonomy
Cognition	Cross-session memory poisoning; instruction-data confusion in long, multi-source context windows Cross-tenant memory contamination; weak edge-side summarisation or routing exploited as a chain weak link	Cross-session RAG and long-horizon memory cannot scale without per-entry provenance or costly write-side audits Shared memory pools across agents or business units are blocked; teams revert to siloed stores and manual reconciliation
Execution	Narratives that diverge from ground-truth tool traces; irreversible actions without compensating controls Privilege creep and malicious-by-construction tools composed into high-impact effects	Regulated and embodied deployments lack high-assurance automation without signed, replayable execution artefacts Dynamic tool composition is disallowed or confined to narrow sandboxes, preventing end-to-end agentic workflows

Identity gaps. Most frameworks treat identity as self-declared names or API keys, not as cryptographic evidence binding builders, binaries or prompts, and authorized capability sets [34, 6]. That gap enables impersonation and *supply-chain* injection of third-party skills, plugins, and tools whose provenance cannot be checked [35]. In GUI- and multimodal-driven deployments, presentation-layer spoofing (package names, icons, screenshots) further blurs “which application or principal the agent is acting for,” coupling identity failures to downstream perception and execution errors.

Perception gaps. Direct and indirect prompt injection remains the dominant vector [32]: without mandatory mediation, tool outputs, web pages, files, and agent-to-agent messages are ingested as unlabeled high-entropy text indistinguishable from operator instructions. *State deception*—transparent overlays, adversarial screenshots, or noisy sensor streams—poisons the agent’s world model even when syscall policies are satisfied, a failure mode highlighted by safety analyses of computer-use agents [33]. Operationally, teams respond by stripping multimodal context or inserting human-in-the-loop review, which caps automation even when no attacker is present.

Cognition gaps. Persistent stores let a single poisoned entry influence later sessions [36, 37], analogous to a TOCTOU class of bug where the unsafe act (retrieval) is separated in time from the poisoning write. Long contexts assembled from heterogeneous sources exacerbate instruction–data confusion, biasing planning and tool selection without surfacing low-confidence branches to users. Heterogeneous deployments that route summarisation, filtering, or delegation through resource-constrained edge components further concentrate risk on the weakest semantic link in the chain [38].

Execution gaps. Agents often hold broad, slowly changing tool permissions while tasks demand fine-grained, ephemeral authority; bridging that gap without telemetry invites *privilege creep* and composition of nominally benign tools into high-impact sequences. Malicious or trojaned skills and scripts remain dangerous even when the LLM “behaves,” because execution is ultimately local code with ambient access. The *plan–trace alignment* problem—natural-language rationales that drift from signed execution records—collapses auditability for regulated and irreversible (e.g., financial or physical) actions; recent systems mitigate this class of failure by cross-checking claims against signed tool receipts [39].

Collectively, these gaps call for a *single semantic mediation layer* that is mandatory for the agent lifecycle rather than bolted on per integration point. Section 3 states the design principles we adopt for such a layer, and Section 4 presents AGENTKERNEL as a concrete architecture that instantiates them.

3 Design Principles

Before presenting AGENTKERNEL’s architecture, we distill five design principles that guide every aspect of the system. These principles are not merely implementation guidelines; they constitute the intellectual core of our position and distinguish AGENTKERNEL from capability-first agent OS proposals.

Principle 1: Security as an Independent & Mandatory Trusted Layer. The agent security kernel must operate as an *external, non-bypassable reference monitor*—independent of the agent’s own reasoning loop.

In classical OS security, the reference monitor concept [40] requires that every access be mediated by a tamper-proof authority that is itself not subject to the policy it enforces. The same principle applies to agents, but with a critical twist: the agent’s “reasoning loop” (the LLM) is inherently manipulable through prompt injection. Embedding security logic *inside* the LLM’s context—as many guardrail approaches do—means that a successful injection can simultaneously subvert the security check and the protected action. AGENTKERNEL therefore places its security kernel *outside* the LLM context, mediating all inputs and outputs through an independent enforcement layer. The agent cannot bypass the kernel because the kernel controls the only interfaces (adapters) through which the agent can reach the LLM, tools, and storage.

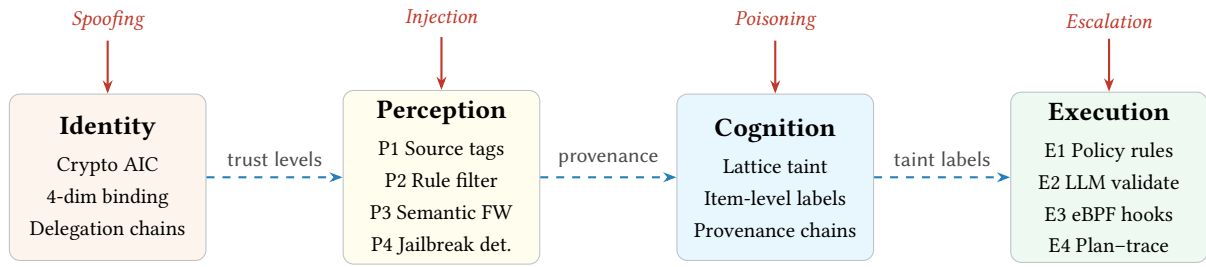
Principle 2: Deny-by-Default with Policy Intersection. When multiple policies apply, the effective permission set is the *intersection*—never the union—of all applicable policy sets.

In multi-stakeholder agent deployments, permissions originate from several sources: the developer declares a maximum capability set at build time, the operator grants a runtime subset, and the agent security kernel further constrains based on context. If the combination rule were a *union*, any single permissive policy could override all others. AGENTKERNEL mandates *intersection*: the effective permission is always

$$S_{\text{eff}} = S_{\text{dev}} \cap S_{\text{ops}} \cap S_{\text{ctx}}$$

This “converge-on-strictest” rule applies at every level: AIC capability chains (§4.2), skill permission composition (§4.5), and cross-agent session tokens (§4.2).

Principle 3: Defense-in-Depth Across the Agent Lifecycle. Each of the four pillars (Identity, Perception, Cognition, Execution) provides *independent, composable* defense layers; compromise of one



Each pillar independently reduces residual risk; compromise of one does not cascade to others.

Figure 3: Defense-in-depth across the agent lifecycle. Each pillar independently intercepts a distinct threat class. Provenance labels flow across pillars (dashed arrows), enabling downstream pillars to make informed decisions based on upstream assessments. Compromise of one pillar does not cascade to others.

pillar does not cascade to others.

Defense-in-depth is a well-established security doctrine, yet most agent security approaches implement it *within* a single layer (e.g., multiple prompt-injection filters in sequence). AGENTKERNEL extends defense-in-depth *across the entire agent lifecycle*: identity prevents impersonation regardless of whether perception catches an injection; perception sanitizes inputs regardless of whether cognition tracks taint; cognition isolates contaminated memory regardless of whether execution blocks the resulting action. Each pillar reduces the residual risk left by the others, and their independence means that a zero-day in one does not unravel the entire defense. Figure 3 visualizes this cross-lifecycle defense architecture.

Principle 4: Semantic-Level Mediation. The agent security kernel mediates at the *semantic level* (intents, plans, memory entries) while also leveraging OS-level enforcement (eBPF, process sandboxes) as the ultimate backstop.

Traditional OS security operates at the syscall boundary—it can prevent a process from opening a file but cannot determine whether the file’s *content* was derived from a poisoned memory entry. Agent security requires a higher-level mediation point that understands *what the agent intends* and *why*, not just *which syscall* it issues. AGENTKERNEL achieves this through a dual-layer approach: semantic mediation at the tool-call, memory-access, and plan-evaluation levels (§4.3–4.5), backstopped by kernel-level enforcement via eBPF hooks that constrain the actual system calls (§4.5). This dual-layer design closes the semantic–syscall gap identified in §2.2.

Principle 5: Narrow Agent–Kernel Integration Boundary. Relative to the manipulable agent loop, AGENTKERNEL exposes exactly three narrow integration interfaces—the LLM, tool, and storage adapters—bounding the *agent-visible* surfaces of inference, tool execution, and durable memory (distinct from the kernel’s internal modularization).

The four pillars (Identity, Perception, Cognition, Execution) specify *how* policy is enforced across the lifecycle (Principle 3; Figure 3): composable invariants in the trusted enforcement layer. The three adapters specify *where* untrusted agent and framework code may reach model inference, tools, and durable memory. Pillar services attach along these paths (e.g., perception on inbound model-bound content); identity is a kernel-managed control-plane substrate, not a fourth application-side “identity adapter.” Restricting co-evolving integration glue to three choke points instantiates *economy of mechanism* and *complete mediation*.

for that resource model. This is *not* the claim that “three APIs” implement the pillars, nor that the TCB is small: GAR, the Agent Kernel, and pillar logic remain inside the TCB (§4).

Like Principle 1, soundness requires *non-bypass*: the agent and its orchestration fabric must not retain *parallel* access to the same model endpoints, tool runtimes, or stores outside AGENTKERNEL (ambient credentials, direct tool transports, filesystem/DB paths that bypass the storage adapter, *etc.*). Otherwise mediation is incomplete regardless of adapter narrowness. Deployment shape is secondary—embedded library, gateway, sidecar, or tool facade—provided the mandatory trusted layer remains the sole authority over those resources.

The adapters then yield three integration properties:

- **Scoped complete mediation.** All policy-relevant traffic for the three capability classes traverses the adapters, giving one auditable seam to harden or replace glue code.
- **Kernel–application decoupling.** The kernel can be audited, tested, or upgraded without rewriting agent logic; frameworks can be swapped without re-implementing pillar semantics.
- **Explicit evolution rule.** A new externally visible capability class beyond model, tool, and storage effects warrants a new integration abstraction, not ad hoc bypasses.

Relationship to classical OS principles. These five principles are deeply rooted in classical OS security: Principle 1 extends the reference monitor [40]; Principle 2 generalizes the principle of least privilege and fail-safe defaults [41]; Principle 3 adapts defense-in-depth to the agent lifecycle; Principle 4 bridges the semantic–syscall gap; and Principle 5 applies economy of mechanism and complete mediation [41] at the agent–kernel integration boundary. What is new is their *application to autonomous AI agents*—a domain where the “user” is an LLM, the “application” is an agentic loop, and the “data” may itself contain executable intent.

4 The AGENTKERNEL Architecture

This section presents the AGENTKERNEL architecture at the conceptual and structural level. We begin with the agent–kernel contract and a structural overview grounded in the three-domain model of Section 2, then detail each of the four security pillars.

4.1 Architectural Overview

Traditional OS kernels mediate access to physical resources through a mandatory, non-bypassable reference monitor. Agents, however, fail at the *semantic* boundary: untrusted content, poisoned memory, and mis-attributed actions cross into reasoning and tool execution without a single choke point that can enforce end-to-end policy. We therefore treat an *agent OS kernel* as a semantic reference monitor: every interaction with the outside world must pass through kernel-enforced services that compose into a coherent trust argument.

Concretely, such a kernel must supply four integrated services, each addressing a distinct class of failures that Section 2 isolates across the agent lifecycle:

1. **Cryptographic agent identity** binds builders, the code or configuration the agent actually runs, and authorization to deploy—mitigating impersonation and supply-chain substitution while enabling verifiable agent-to-agent collaboration. Identity material (signing and attestation keys) is held and used only through the kernel, never as an ambient secret in application space.
2. **Graduated input mediation** applies multiple independent filters before content reaches the LLM context, trading off latency for defense in depth against injection, spoofing, and high-entropy confusion—improving safety without forcing operators to discard entire modalities.

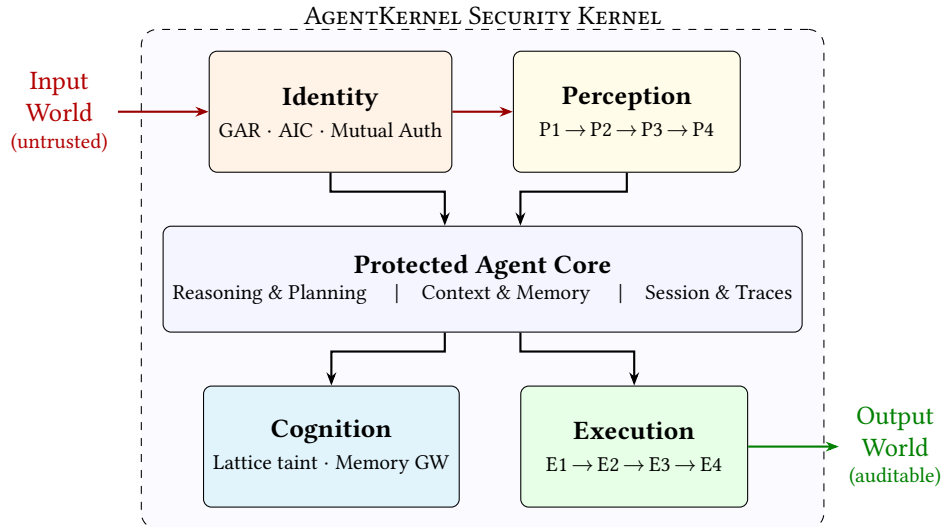


Figure 4: *AGENTKERNEL* architecture. The security kernel mediates all interactions between the untrusted input world and the auditable output world. Four security pillars operate around the protected agent core, each enforcing independent invariants.

3. **Information-flow control** over agent memory tracks provenance and taint at per-entry granularity so retrieval cannot silently amplify stale or adversarial state—supporting both poisoning resistance and provenance-aware cognition.
4. **Execution governance** closes the loop from stated intent through tool plans to syscall-level effects, checking alignment with operator policy and producing replayable evidence when actions are sensitive or irreversible—restoring audibility while still allowing dynamic tool expansion under explicit policy.

AGENTKERNEL is our instantiation of this contract. It is realized as a *mandatory enforcement boundary* wrapped around the agent core: every interaction—inbound content, outbound actions, agent-to-agent communication—must traverse the security kernel, with each pillar enforcing independent invariants. Figure 4 illustrates the overall architecture.

The architecture is governed by two standalone trusted services that operate outside the agent’s control:

- The **Global Agent Registry (GAR)**—a remote, CA-grade service responsible for developer enrollment, Agent Identity Card (AIC) issuance via Ed25519 signatures, policy governance, and revocation.
- The **Agent Kernel**—a local trust anchor that holds the agent’s private key (never exported), mediates mutual attestation, manages OIDC secrets, and hosts pluggable security modules for each pillar.

The agent application connects to *AGENTKERNEL* through exactly three narrow adapter interfaces (Principle 5)—bounding the agent-visible surfaces of model inference, tool execution, and memory I/O: the LLM adapter, the tool adapter, and the storage adapter. This confined integration boundary ensures that the security kernel can be independently audited and replaced without modifying agent logic.

4.2 Pillar 1: Identity as a Kernel-Managed Resource

Key Insight 1. Agent identity must be a kernel-managed resource—held, protected, and mediated by the security kernel, never exposed to agent application code. Just as an OS kernel manages process credentials on behalf of user-space programs, the agent security kernel manages cryptographic identity on behalf of agents.

In existing agent frameworks, identity is a self-declared string—an agent name or an API key—that lives in the agent’s own configuration and is fully accessible to application code. This conflation of identity *holder* and

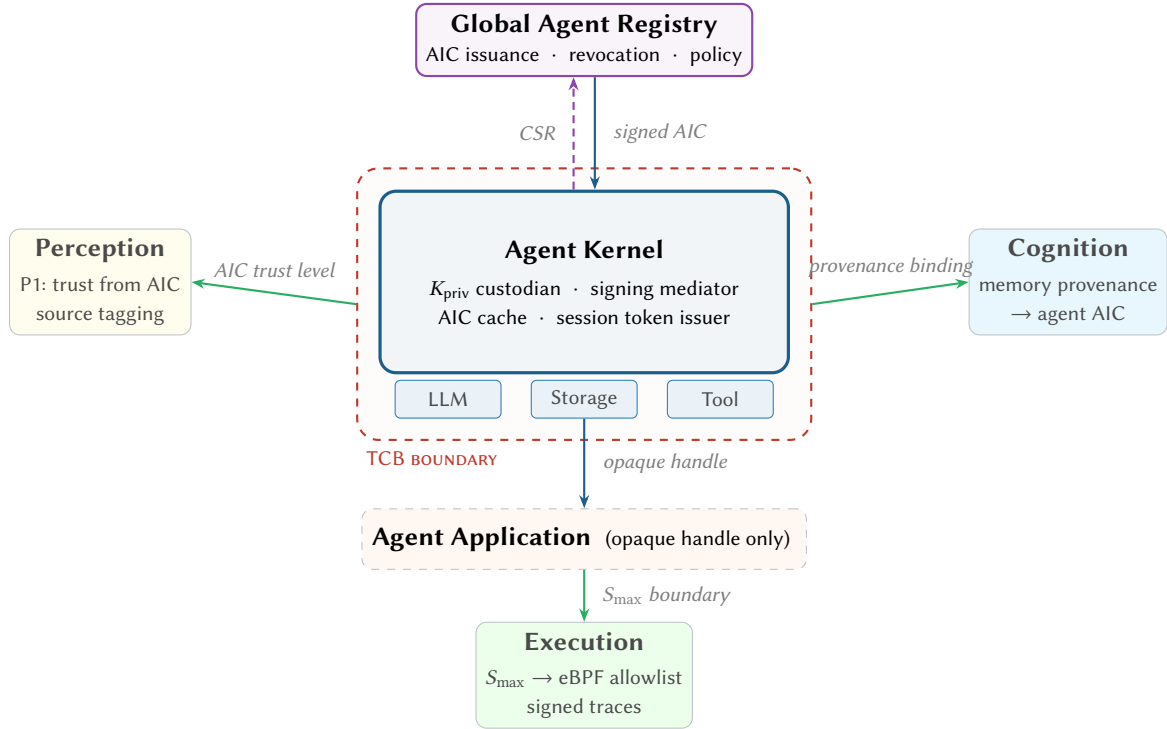


Figure 5: Identity as a kernel-managed resource. The Agent Kernel holds the private key and mediates all identity operations on behalf of the agent. Identity-derived trust signals flow into each of the other three security pillars through the kernel’s control plane, making identity the shared trust substrate for the entire security architecture.

identity *subject* is the root cause of agent impersonation, unauthorized delegation, and supply-chain attacks: the agent that uses a credential is the same entity that stores it, so compromising the agent automatically compromises the identity.

AGENTKERNEL breaks this conflation by treating identity as a *kernel-managed resource*, analogous to how an OS kernel manages process UIDs and capabilities. The agent’s private key K_{priv} resides exclusively within the Agent Kernel—a local trust anchor that acts as an HSM-like custodian. Agent application code never touches raw key material; it interacts only with opaque signing handles. All cryptographic operations—AIC presentation, challenge signing, session token issuance—are *mediated by the kernel*, not performed by the agent itself. Figure 5 illustrates this kernel-mediated identity architecture.

Provisioning: what the kernel binds. At provisioning time, the Agent Kernel generates a fresh Ed25519 keypair, constructs an Agent Identity Card (AIC) that cryptographically binds four dimensions—developer, code artifact, operator, and deployment context—into a single verifiable credential [42], and submits it to the Global Agent Registry (GAR) for signing. The signed AIC is cached locally; the private key never leaves the kernel boundary. The four-dimensional binding model builds on our earlier work on cryptographic agent identity [43]; here we focus on how the kernel *uses* this identity to drive OS-level enforcement.

TEE-backed key custody. Identity-key storage follows a *tiered* custody path—from file- and OS-keychain-backed handles up to an optional trusted execution environment (TEE) tier (e.g., Intel SGX, ARM TrustZone, or Nitro-class enclaves) [44, 45]. When the TEE tier is used, the identity private key is provisioned and used *inside* the enclave so raw material never crosses the hardware isolation boundary; the Agent Kernel still mediates signing through non-exportable primitives, preserving the same kernel-controlled interface to agent code while strengthening extraction resistance against a compromised host OS.

Identity-driven enforcement. In a traditional OS, a process’s UID determines which files it can open, which signals it can send, and which system calls it can invoke. In AGENTKERNEL, the AIC’s *capability boundary* $S_{\max}$ plays an analogous role: it is the structural input to every enforcement decision across the security kernel. At the execution pillar, Layer E1 cross-references each tool invocation against the intersection of the agent’s $S_{\max}$ and the tool’s declared permission manifest; Layer E3 translates this intersection into an eBPF allowlist installed before the tool’s process tree is spawned. When multiple skills compose within a single agent, the kernel computes the *intersection* of all skill permission envelopes with the agent’s $S_{\max}$ (Principle 2)—ensuring that composition can only narrow, never widen, the effective capability set.

Cross-pillar identity integration. Unlike protocol-level identity systems that operate in isolation, AGENTKERNEL’s kernel-managed identity serves as a *shared trust substrate* consumed by all four security pillars:

- **Perception.** Layer P1 (Source Tagger) assigns baseline trust levels partly based on whether the content source carries a valid AIC: messages from AIC-verified agents receive higher initial trust than content from unverified origins, reducing perception overhead for trusted peers.
- **Cognition.** Every memory item’s provenance chain traces back to the originating agent’s AIC, enabling the memory gateway to enforce label-based access control—an agent can retrieve only items whose provenance labels are compatible with its own identity and clearance level.
- **Execution.** As described above, the AIC’s capability boundary is the structural ceiling for all tool-call permissions and eBPF allowlists. Additionally, execution traces are signed by the kernel using the agent’s K_{priv} , binding every recorded action to a non-repudiable identity.

This cross-pillar integration is the key architectural difference between AGENTKERNEL’s identity pillar and standalone agent identity protocols: identity is not merely an authentication credential exchanged at session start, but a *live enforcement input* that constrains every security decision throughout the agent’s lifecycle.

Kernel-mediated trust establishment. When two agents need to interact, neither agent touches the cryptographic handshake directly. Instead, the Agent Kernel orchestrates the entire trust negotiation: it generates fresh nonces, verifies both parties’ AICs against the GAR trust store, signs challenges using the kernel-held K_{priv} , and issues a scoped session token whose capability boundary is the intersection $S_{\text{session}} \subseteq S_{\max}^A \cap S_{\max}^B$. The token is context-bound and lifecycle-ephemeral—it expires when either agent terminates, preventing credential leakage. Because all signing is kernel-mediated, a compromised agent cannot forge attestations or escalate its own token scope, just as a compromised user-space process cannot forge kernel-issued capabilities.

Delegation as a kernel operation. In multi-agent orchestration, spawning a sub-agent with delegated authority is a *kernel system call*, not an application-level action. The kernel verifies that the child’s requested capabilities are a strict subset of the parent’s $S_{\max}$, enforces a configurable depth ceiling, and issues a child AIC whose validity is capped to the parent’s remaining lifetime. Revoking a parent AIC atomically invalidates the entire delegation subtree—cascading revocation is enforced structurally by the kernel, not by policy.

4.3 Pillar 2: Perception

Key Insight 2. The best defense intercepts threats before the LLM ever sees them. Defense-in-depth should apply across layers of increasing cost and sophistication.

LLM-powered agents ingest content from inherently untrusted sources. The perception pillar establishes a *graduated defense pipeline* that processes all input before it enters the LLM’s context window. Unlike

single-point defenses, AGENTKERNEL’s perception is designed as *layered depth*: each layer catches what the previous one misses, with cost and sophistication increasing at each stage.

Layer P1—Source Tagger. Every piece of inbound content is tagged with a *trust level* based on its origin: SYSTEM (fully trusted), USER (high trust), TOOL (conditional), EXTERNAL (low—boundary markers added), or UNTRUSTED (flagged by downstream filters). These tags generate *ProvenanceTag* metadata (origin, timestamp, trust level, detection warnings) that persist through the entire pipeline and are consumed by downstream cognition and execution pillars.

Layer P2—Rule-Based Filter. A high-speed pattern-matching layer intercepts known injection templates: instruction overrides (“ignore previous instructions”), role impersonation, encoding bypasses (Base64, Unicode obfuscation), and token-stuffing payloads. Rules are sub-millisecond, deterministic, and auditable, handling the majority of known attack patterns at near-zero cost.

Layer P3—Semantic Firewall. For content that passes rule-based filters but may contain semantically adversarial intent, a lightweight LLM classifier evaluates whether the content attempts to manipulate agent behavior, override system instructions, or present intent inconsistent with its declared type. The classifier operates as a binary decision gate with context-aware thresholds—stricter for low-trust sources identified by P1.

Layer P4—Jailbreak and Multi-Turn Detector. Sophisticated attacks unfold across multiple conversation turns [46]. P4 analyzes conversation history to detect gradual role redefinition, progressive safety degradation, statistical deviations from normal flow, and known jailbreak templates.

Integration with identity. Provenance tags inherit trust levels from verified agent identities: content from agents with valid AICs receives higher baseline trust than content from unverified sources. This creates a virtuous cycle where strong identity reduces perception overhead for trusted agents.

4.4 Pillar 3: Cognition

Key Insight 3. Memory is not just the agent’s persistent context—it is a security boundary. Information flow control theory, applied at entry granularity, can preserve usability while preventing contamination.

As agents gain long-term memory and multi-agent collaboration capabilities, memory becomes a critical security surface. A single piece of untrusted content persisted in memory can *persistently influence behavior across sessions*—far more dangerous than transient prompt injection.

AGENTKERNEL’s cognition pillar treats memory as a *managed, labeled, auditable data plane* with information-flow guarantees, structured across four layers: Agent Host, Memory Gateway, Memory Control Plane, and Storage.

Lattice-based entry-level taint propagation. The core algorithmic innovation is *item-level label propagation on a product lattice*, rather than coarse-grained “entire session inherits the worst label” approaches.

Consider a session containing: a user preference ($trusted \times public$), a private email ($trusted \times secret$), and a web excerpt ($untrusted \times public$). Naively taking the join produces $untrusted \times secret$ for the *entire* session—rendering most memory unusable.

AGENTKERNEL instead operates at the individual memory item level. The process follows five steps: (1) segment the conversation into source segments with known provenance; (2) extract candidate memory items; (3) for each candidate, determine which source segments actually support it; (4) search the label lattice for the minimum security label that still supports the item’s validity (utility loss $\leq \lambda$); (5) output a set of incomparable minimal labels (a minimal anti-chain), not a single worst-case collapse.

This approach—rooted in classical lattice models of information flow [47, 48, 49]—preserves memory usability while maintaining provenance precision.

Cross-agent memory sharing. AGENTKERNEL adopts a “default private, explicit sharing” principle. Memory is private by default; sharing requires explicit authorization (shared_with lists, task-scoped workspaces, or human-approved export). Memory with high-sensitivity labels is *never automatically shared* across agent boundaries. Even shared memory maintains independent access audit trails per agent.

Anti-injection for memory. Historical context retrieved from memory is explicitly marked as *non-executable data*—it cannot be interpreted as instructions. Combined with provenance chains that trace every memory entry to its origin, this prevents the “memory-as-instruction” attack pattern.

4.5 Pillar 4: Execution

Key Insight 4. From intent to syscall, every layer must enforce least privilege. Claimed actions must be verified against actual execution traces.

The execution pillar bridges the gap between *what the model says* and *what the system actually does*, through four layers that collaborate to ensure integrity.

Layer E1—Rule/Policy Evaluator. At the tool invocation boundary, fast deterministic checks enforce deny-lists, capability alignment (cross-referencing the tool’s manifest with the agent’s AIC boundary), path/command/domain constraints, and deny-first conflict resolution. Cost: sub-millisecond, no LLM call.

Layer E2—LLM Validator. For high-ambiguity or high-impact decisions where rules alone are insufficient, a lightweight LLM evaluates intent alignment (“Is this action consistent with the user’s stated goal?”) and may trigger human-in-the-loop confirmation. Invoked only when E1 escalates.

Layer E3—eBPF/Kernel Hook Manager. This is AGENTKERNEL’s most distinctive execution innovation. Rather than trusting tool implementations to respect declared constraints, a *privileged guardian daemon*: (1) pre-computes an allowlist before each tool execution (which file paths, network endpoints, and DNS queries are permitted); (2) installs the allowlist in kernel-level eBPF probes or system hooks; (3) monitors the entire process tree spawned by the tool—child processes inherit the same constraints; and (4) pauses violating processes immediately, then applies remediation. Even if a tool’s implementation is malicious, it *cannot* access resources outside the pre-computed allowlist.

Layer E4—Plan-Trace Aligner. Post-execution verification aligns what was planned with what actually happened. Every tool call’s parameters, timing, and result are recorded as an immutable trace. The aligner compares the trace against the agent’s stated plan, surfacing missing steps, extra steps, reordered steps, and—critically—*hallucinated actions* where the LLM claims to have performed operations that the trace shows never occurred.

Skill permission manifests & workflow orchestration. Tools and skills declare their required system surfaces (file system, network, execution, messaging) via structured manifests. When multiple skills compose into a workflow, AGENTKERNEL computes the *intersection*, not the union, of their permission sets (Principle 2). The declaration layer determines “how far a skill can reach at most”; the runtime layer (E1–E3) determines “whether this specific invocation can actually happen.” This structural permission convergence provides a secure substrate for *tool and workflow orchestration*: orchestrators can freely chain tools and delegate sub-tasks, knowing that the kernel will automatically bound the workflow’s authority to the safest common denominator.

Agent scheduling & concurrency isolation. As agents scale to handle concurrent workflows, AGENTKERNEL provides the necessary isolation primitives for *agent scheduling and concurrency*. Consistent with the identity pillar (§4.2), each concurrent execution context is cryptographically bound: local concurrent threads receive distinct, lifecycle-ephemeral execution tokens, while delegated sub-agents operate under their own child AICs. At the execution layer, Layer E3 enforces concurrency isolation by mapping each execution token or child AIC to a dedicated BPF namespace and process tree. This ensures that concurrent agent tasks—even those running the same skill—cannot interfere with each other’s memory, file descriptors, or network sockets, enabling safe, high-throughput agent scheduling.

Deterministic-first philosophy. The main execution path uses deterministic allowlists and kernel enforcement; LLM adjudication (E2) is an optional enhancement for edge cases. This ensures that security does not degrade when the LLM makes errors or is manipulated.

5 Security Analysis

A security architecture is only as strong as the argument that supports it. This section states the threat model and trust assumptions under which AGENTKERNEL’s guarantees hold (§5.1), derives precise security invariants for each pillar and argues their soundness (§5.2), examines how the four pillars compose into end-to-end guarantees (§5.3), analyzes three architectural choices that fundamentally distinguish AGENTKERNEL’s security model from prior approaches (§5.4), and closes with an honest accounting of residual risks (§5.5).

5.1 Threat Model and Trust Assumptions

AGENTKERNEL targets a strong adversary model in which the attacker may control inputs, supply compromised tools, and exploit misconfigurations, but cannot break standard cryptographic assumptions or bypass the mandatory mediation boundary.

Attacker capabilities. We assume an adversary with the following capabilities:

1. **Input manipulation.** The attacker can craft arbitrary inputs to the agent—prompt injection in tool outputs, web pages, files, and inter-agent messages; jailbreak attempts; encoding bypasses; and multi-turn conversational attacks. These attacks target the perception and cognition pillars.
2. **Tool compromise.** The attacker can supply a malicious tool that passes semantic-level parameter inspection but attempts unauthorized operations at runtime—spawning child processes, accessing files outside declared scope, or opening network connections to exfiltration endpoints. This targets the execution pillar.
3. **Memory poisoning.** The attacker can inject content into agent memory through normal interaction channels, aiming to influence retrieval and decision-making in later sessions (§2.3). This targets the cognition pillar.
4. **Impersonation and replay.** The attacker can attempt to masquerade as a legitimate agent or operator using replayed credentials, self-declared identity strings, or stolen API keys.

5. **Supply-chain injection.** The attacker can publish malicious skills or plugins that declare deceptively narrow permissions while containing hidden malicious functionality.
6. **Operator misconfiguration.** A benign but mistaken operator may set overly permissive policies. AGENTKERNEL’s structural guarantees are designed to survive this case (§5.4.1).

Attacker limitations and trust assumptions. The following capabilities are assumed *outside* the attacker’s reach; violating any of them voids the corresponding guarantees:

1. The attacker **cannot forge Ed25519 signatures** without access to the corresponding private key, nor break standard cryptographic assumptions (hash collision resistance, etc.).
2. The attacker **cannot extract private keys from the Agent Kernel.** The kernel holds K_{priv} and mediates all signing operations; agent application code never touches raw key material (§4.2).
3. The attacker **cannot bypass the three adapter interfaces** (LLM, tool, storage). All agent-visible paths to model inference, tool execution, and durable memory must traverse these adapters (Principle 5). If the agent retains ambient credentials or direct filesystem access outside the adapters, mediation is incomplete.
4. The attacker **cannot modify eBPF probes** without OS kernel-level privilege. The host OS kernel is assumed to correctly enforce the allowlists installed by Layer E3 (§4.5).
5. The **GAR signing key is assumed uncompromised.** GAR is the root of trust for agent identity; its compromise would enable arbitrary AIC forgery and is therefore catastrophic for the identity pillar.

Explicitly out of scope. Physical attacks, side-channel attacks (timing, power analysis), and compromise of the host OS kernel itself are outside the threat model. Attacks that exploit LLM reasoning errors without violating a stated invariant (e.g., the agent making a poor but policy-compliant decision) are reliability issues rather than security violations in our model.

Trusted computing base. The TCB consists of the Global Agent Registry (GAR), the Agent Kernel, the three adapter interfaces, the OS kernel’s eBPF subsystem, and the cryptographic primitives (Ed25519, hash functions). Each component is described in §4; their correctness is the foundation on which the following invariants rest.

5.2 Security Invariants

For each pillar we state precise security invariants, identify the mechanism that enforces them, and give a brief soundness argument. Table 2 provides a consolidated summary.

Identity invariants. The identity pillar provides the cryptographic substrate on which all other pillars depend. I1 (AIC unforgeability) follows from Ed25519 security and the fact that GAR is the sole authority for AIC issuance; an agent cannot self-certify an identity. I2 (delegation monotonicity) is the structural guarantee that distinguishes AGENTKERNEL’s capability model: the kernel enforces $S_{max}^{child} \subseteq S_{max}^{parent}$ by construction, and no policy override can widen the child’s boundary. I3 and I4 are standard cryptographic properties (non-repudiation and freshness) realized through kernel-mediated signing and single-use nonces respectively.

Perception invariants. P1 is a *mediation completeness* property: every unit of content that reaches the LLM context must carry a provenance tag. It holds because all input paths (user messages, tool outputs, web content, inter-agent messages) are routed through the graduated perception pipeline, where P1 (Source Tagger) is the mandatory first stage. P2 is a *defense-in-depth* property: the four layers P1–P4 operate on orthogonal features, so an attacker who evades one layer (e.g., crafting a payload that passes syntactic filters) still faces independent barriers at the semantic and temporal layers.

Cognition invariants. C1 is an *information-flow soundness* property. The lattice-based search at item granularity (§4.4) ensures that the taint label assigned to each memory item dominates the labels of all

Table 2: Security invariants of the AGENTKERNEL architecture.

Pillar	Invariant	Soundness Argument
Identity	I1. AIC unforgeability	Ed25519 signatures verified against GAR root at every presentation; K_{priv} never leaves kernel boundary (§4.2).
	I2. Delegation monotonicity: $S_{\text{max}}^{\text{child}} \subseteq S_{\text{max}}^{\text{parent}}$	Kernel verifies subset relation before issuing child AIC; child validity is capped to parent’s remaining lifetime.
	I3. Action non-repudiation	All execution traces kernel-signed by K_{priv} ; signing is kernel-mediated, never performed by agent code.
	I4. Session anti-replay	Fresh nonces with TTL per session; context-bound tokens; lifecycle ephemerality (§4.2).
Perception	P1. Provenance completeness: all content entering LLM context carries origin tags	All input paths traverse P1 (Source Tagger) as mandatory first stage; tags persist through P2–P4.
	P2. Layered independence: bypassing one filter does not help bypass others	P1–P4 operate on orthogonal features (source identity, syntactic patterns, semantic intent, multi-turn history).
Cognition	C1. Taint soundness: derived items inherit dominating labels from source segments	Lattice search identifies all supporting segments and computes minimal dominating label (§4.4).
	C2. Default-private sharing	Memory gateway enforces explicit authorization on every cross-agent access; no implicit sharing.
Execution	E1. Intersection least-privilege: $S_{\text{eff}} = \bigcap_i S_i$	Kernel computes intersection at each composition point (Principle 2); deny-by-default resolution.
	E2. eBPF non-bypassability: no process exceeds dynamically installed allowlist	eBPF probes installed before process spawn; child processes inherit constraints (§4.5, E3).
	E3. Plan–trace alignment: deviations between declared plan and execution are detected	E4 records immutable traces and compares against stated plan; hallucinated actions surface as mismatches.

source segments from which it was derived. Critically, this guarantee depends on the accuracy of source segmentation (step 1 of the algorithm); adversarially crafted content designed to blur segment boundaries can degrade label fidelity—a limitation we address in §5.5. C2 enforces that cross-agent memory sharing is explicit and auditable, preventing the “contaminated memory propagates silently” failure mode identified in §2.3.

Execution invariants. E1 is the most architecturally consequential invariant: it guarantees that policy composition can only *narrow* permissions. Unlike systems where multiple policy sources are combined via union or priority ordering, AGENTKERNEL’s intersection semantics ensure that no single permissive policy can override a restrictive one. E2 bridges the gap from semantic permission to syscall enforcement: the dynamically computed allowlist is installed as eBPF probes before tool execution, and the entire process tree inherits the constraints. E3 provides *detection* rather than prevention: it does not block plan–trace mismatches but guarantees they are surfaced for audit, enabling retrospective accountability even when preventive controls fail.

5.3 Composition and Cross-Pillar Guarantees

The four pillars are not isolated; they share provenance metadata that flows through the system along a defined chain: Identity establishes *who* produced content; Perception assigns initial *trust levels*; Cognition propagates *taint labels* through memory derivations; Execution consumes these labels to build *allowlists* and constrain tool actions. This flow creates both composition benefits and coupling risks, which we analyze in turn.

Layered defense. The key composition argument is that an attacker must defeat *multiple independent mechanisms* to achieve end-to-end compromise. Consider the motivating DevOps scenario from §1: to successfully push malicious code, an attacker must (i) evade identity checks to impersonate a legitimate delegator, (ii) bypass perception filters to deliver the injection payload, (iii) overcome taint propagation in cognition to have the poisoned summary treated as trusted context, and (iv) execute tool actions that pass both semantic alignment (E1–E2) and syscall enforcement (E3). Each pillar independently reduces the residual attack surface: even if perception misses an injection, cognition’s taint labels still flag the resulting memory entry; even if cognition propagates contaminated labels, execution’s eBPF allowlist remains constrained by the agent’s $S_{\max}$ and the tool’s declared manifest.

Coupling and independence. We explicitly do *not* claim that the four pillars are fully independent—they share provenance metadata, so a failure in an upstream pillar can affect downstream decisions. If identity verification is compromised (I1 violated), perception assigns incorrect trust levels; if source segmentation fails, cognition computes incorrect taint labels; if taint labels are wrong, execution may construct overly permissive allowlists. What the architecture guarantees is not independence of *outcomes* but independence of *enforcement mechanisms*: each pillar independently re-validates upstream claims against its own invariants rather than blindly trusting upstream metadata. Perception cross-checks identity claims; cognition cross-checks provenance tags against source segments; execution cross-checks taint labels against declared manifests. This *re-validation chain* means that an upstream failure must survive downstream scrutiny to become an end-to-end compromise—a higher bar than a single-point failure.

Why four pillars, not one. A natural question is whether a single unified policy engine could achieve the same guarantees with less complexity. The answer lies in the *heterogeneity of the attack surface*: identity is a cryptographic problem; perception is a classification problem; cognition is an information-flow problem; execution is a mediation problem. Each demands different mechanisms, failure models, and verification techniques. Unifying them into a single layer would force a least-common-denominator approach that weakens guarantees at every boundary. The four-pillar decomposition is not an aesthetic choice; it follows from the structure of the threat surface itself.

5.4 Architectural Distinctions

Three architectural choices in AGENTKERNEL yield security properties that are unattainable under conventional designs. We analyze each as a security argument rather than a design rationale.

5.4.1 Structural vs. Policy-Based Least-Privilege

Most agent frameworks enforce least-privilege through policy engines (OPA, Cedar, XACML). Policy-based approaches share a structural weakness: they *fail open* on misconfiguration. An overly permissive policy silently grants excessive access, and the system has no independent means to detect the error because the policy is the authority.

AGENTKERNEL inverts this failure mode through cryptographically enforced monotonic attenuation. At each stage—developer declaration, operator provisioning, GAR issuance, runtime enforcement—the effective capability set is the *intersection* of the current stage’s grant with the cryptographically signed boundary from the preceding stage:

$$S_{\max}^{\text{child}} \subseteq S_{\max}^{\text{parent}} \quad (\text{enforced by signature verification, not policy evaluation})$$

No policy change at any single stage can grant capabilities beyond the preceding stage’s signed envelope. We state this as a structural guarantee:

Property 5.1 (Monotonic Capability Attenuation). For any delegation chain $A_0 \rightarrow A_1 \rightarrow \dots \rightarrow A_n$, the effective capability set satisfies $S_{\max}^{A_n} \subseteq S_{\max}^{A_{n-1}} \subseteq \dots \subseteq S_{\max}^{A_0}$. Policy can *deny* within this envelope but never *grant* beyond it.

The guarantee follows from two architectural invariants: (i) the kernel verifies the subset relation $S_{\max}^{\text{child}} \subseteq S_{\max}^{\text{parent}}$ against the parent’s signed AIC before issuing a child AIC, and (ii) runtime enforcement (E1) further constrains effective permissions to the intersection of all applicable policies. Even a misconfigured operator policy cannot override invariant (i) because it is enforced at the cryptographic layer, not the policy layer.

The practical implication is significant: in a multi-stakeholder deployment, no single stakeholder can unilaterally escalate privileges. This is the property that enables safe cross-organizational agent delegation—a capability that policy-only approaches cannot provide without a trusted enforcement layer external to the policy engine.

5.4.2 Item-Level vs. Session-Level Taint

Classical information-flow control [47] operates at the granularity of containers (files, processes). When applied to agent memory, the natural container is the *session*: all items derived from a session inherit its worst-case label. This approach is *sound* (no false negatives) but *unusable* (excessive false positives)—a single untrusted web excerpt taints the entire session’s memory, rendering most items inaccessible.

AGENTKERNEL resolves this tension by operating at *item granularity* on a product lattice (integrity $\times$ confidentiality). The algorithm (§4.4) identifies the *minimal* label that preserves each item’s validity, producing a set of incomparable minimal labels (an anti-chain) rather than a single worst-case join. The security question is whether item-level granularity preserves soundness.

The soundness of item-level taint rests on the accuracy of source segmentation (identifying which source segments support each candidate memory item). When segmentation is correct, the item-level label dominates all contributing sources by construction (invariant C1). When segmentation is imperfect—as can happen with adversarially crafted content designed to blur provenance boundaries—the label may under-approximate the true taint. This is a *false-negative risk* that session-level taint avoids entirely.

The architectural trade-off is therefore: session-level taint is sound but yields a degenerate lattice (one label per session), making information-flow control impractical for interactive agents; item-level taint trades a bounded amount of soundness (under adversarial segmentation) for dramatically improved usability. AGENTKERNEL accepts this trade-off explicitly, with the understanding that the execution pillar (E2–E3) provides an independent backstop: even if cognition under-labels a memory item, the eBPF allowlist derived from the agent’s $S_{\max}$ and the tool’s manifest remains in force.

5.4.3 The Semantic–Syscall Bridge

The execution pillar addresses a fundamental architectural gap that neither semantic nor syscall enforcement can close alone:

- **Semantic-only enforcement is bypassable.** A malicious tool that passes parameter inspection can still spawn child processes or access resources outside its declared scope—the tool’s implementation is a black box at the semantic level.
- **Syscall-only enforcement is semantically blind.** It cannot distinguish a file write driven by legitimate reasoning from one driven by a poisoned memory entry—both are identical at the syscall boundary.

AGENTKERNEL bridges this gap through a two-phase protocol: (i) semantic mediation (E1–E2) evaluates the agent’s intent against its identity, input provenance, memory taint, and declared manifests, producing a dynamically computed allowlist; (ii) this allowlist is installed as eBPF probes (E3) before the tool’s process

tree is spawned, constraining the actual syscalls that can be issued. The semantic layer contributes *intent-awareness* (why is this action being taken?); the kernel layer contributes *non-bypassability* (the tool cannot escape the allowlist even if its implementation is malicious).

We state the resulting guarantee:

Property 5.2 (Semantic–Syscall Closure). For any tool invocation t with declared manifest M_t and agent capability boundary $S_{\max}$, the set of syscalls issued by t and its descendants is constrained to $\text{Allow}(t) = \text{SemanticEval}(t, S_{\max}, M_t, \text{taint})$. No process in t 's tree can issue a syscall outside $\text{Allow}(t)$, regardless of t 's implementation.

This guarantee closes the gap identified in §2.2 and is, to our knowledge, unique among agent security architectures. Existing systems address one side or the other: sandboxes provide non-bypassability without intent-awareness; guardrails provide intent-awareness without non-bypassability. AGENTKERNEL's two-phase protocol is the architectural mechanism that composes both properties.

5.5 Residual Risks and Limitations

A credible security analysis must acknowledge what remains unprotected. We enumerate the residual risks that fall outside AGENTKERNEL's current guarantees.

Taint fidelity under adversarial segmentation. The soundness of item-level taint (C1) depends on correct source segmentation in the cognition pillar. Adversarially crafted content designed to blur the boundary between trusted and untrusted segments can cause the lattice search to compute labels that under-approximate true taint. Mitigating this requires robust segmentation techniques, which remain an open research problem (§7).

Semantic firewall accuracy. Perception Layer P3 (Semantic Firewall) is an LLM-based classifier and inherently trades false positives against false negatives. An adversary who understands the classifier's decision boundary can craft payloads in the blind spot. The graduated pipeline (P1–P4) reduces this risk through defense-in-depth but does not eliminate it.

GAR as a single point of trust. The GAR is the root of trust for agent identity; its signing key compromise would enable arbitrary AIC forgery and collapse the identity pillar. Cross-GAR federation (§7) would distribute this trust but introduces its own cross-certification challenges.

Non-bypass assumption. The entire security argument rests on the assumption that the three adapter interfaces are the only paths through which agents can reach LLMs, tools, and storage (Principle 5). If an agent retains ambient credentials, direct filesystem access, or alternative tool transports outside the adapters, mediation is incomplete. Enforcement of this assumption is a deployment responsibility.

eBPF portability. Execution invariant E2 depends on eBPF, which is natively supported on Linux but has limited availability on macOS and Windows (§7). On non-Linux platforms, alternative backends must provide equivalent non-bypassability guarantees.

Formal verification deferred. The invariants stated in §5.2 are enforced by construction but have not been mechanically verified. Formal proofs (e.g., in Coq or TLA+) of the monotonic attenuation chain and the policy intersection framework would elevate these guarantees from architectural claims to machine-checked theorems—work we have deferred to companion publications (§7).

These limitations do not invalidate AGENTKERNEL's architectural contributions, but they bound the confidence with which the stated guarantees should be interpreted. Each limitation corresponds to an active direction in our research agenda.

Table 3: Comparison of representative systems across systematic Agent OS dimensions. Rather than treating security and capability as isolated features, AGENTKERNEL demonstrates how OS-level security primitives (e.g., identity, taint tracking) directly enable advanced agent capabilities (e.g., cross-org A2A, provenance-aware retrieval). ● = comprehensive support as a core design element; ○ = partial or optional support; — = absent or not a design focus.

Dimension	AGENTKERNEL	ACT [26]	AIOS [22]	OpenFang [23]	SmythOS [24]	Letta [25]	nono [29]
Pillar 1: Identity (§4.2)							
Cryptographic workload identity	●	●	—	○	—	—	○
Multi-dimensional provenance	●	—	—	—	—	—	—
Cross-org A2A attestation	●	○	—	—	—	—	—
Pillar 2: Perception (§4.3)							
Input provenance tagging	●	—	—	○	—	—	—
Graduated input filtering	●	○	—	○	—	—	—
Pillar 3: Cognition (§4.4)							
Context & memory management	●	—	○	○	○	●	—
Memory taint tracking	●	—	—	○	—	—	—
Provenance-aware retrieval	●	—	—	—	—	—	—
Pillar 4: Execution (§4.5)							
Tool & workflow orchestration	●	—	○	●	●	—	—
Agent scheduling & concurrency	●	—	●	○	○	—	—
Syscall-level sandboxing	●	—	—	○	—	—	●
Semantic-to-syscall enforcement	●	—	—	—	—	—	—
Plan-trace alignment	●	—	—	○	—	—	—
Cross-Pillar (Principle 3, §3)							
Cross-lifecycle integration	●	—	—	○	—	—	—

6 Related Work: From Agent Harness to Agent OS

The term “agent harness” has rapidly gained currency as the infrastructure concept for managing the full lifecycle of AI agents—encompassing deployment, monitoring, guardrails, tool orchestration, and multi-agent coordination. A growing ecosystem of projects addresses different slices of this harness. While these efforts are individually valuable, we argue that they collectively reveal a deeper need: agents require not just a stronger harness but an *operating system*—a mandatory, non-bypassable mediation layer that provides identity, perception, cognition, and execution services as integrated kernel functions.

This section is organized as follows. We first present Table 3, which compares six representative systems—drawn from the runtime, governance, and sandbox tiers—against AGENTKERNEL across 15 dimensions that map directly to the four architectural pillars of §4, followed by a dimension-by-dimension methodology that defines each row and justifies every non-trivial rating. §6.1 then surveys the broader harness landscape across four tiers: orchestration frameworks (§6.1), agent runtimes (§6.1.2), governance platforms (§6.1.3), and execution sandboxes—providing the detailed evidence behind the table’s ratings. Orchestration frameworks (e.g., LangChain, AutoGen) are omitted from the table because they do not claim OS-level services along these dimensions; their complementary role is discussed in §6.1. §6.2 distills the cross-tier pattern into the argument for a missing OS layer. §6.3 positions AGENTKERNEL as the agent OS kernel. §6.4 discusses implications for agent-to-agent and human-to-agent interactions, and §6.5 covers additional related areas.

Comparison methodology. Each row in Table 3 corresponds to a concrete capability defined in the AGENTKERNEL architecture (§4). We group them by pillar and briefly define each dimension below.

Pillar 1: Identity (§4.2). *Cryptographic workload identity:* whether agent identity is cryptographically bound to builder, code artifact, operator, and deployment context—not merely a self-declared string or API key. AGT earns • via AgentMesh’s SPIFFE/SVID attestation; OpenFang earns ◦ for Ed25519-signed manifests that do not bind all four dimensions; nono earns ◦ for Sigstore bundle signing. *Multi-dimensional provenance:* whether identity material traces across developer, code, operator, and runtime simultaneously. No compared system binds all four dimensions; AGT’s SPIFFE-based workload identity covers the runtime dimension alone (already credited under cryptographic workload identity above). *Cross-org A2A attestation:* mutual identity verification between agents from distinct organizations. AGT earns ◦ via AgentMesh’s SPIFFE federation; no other system provides this.

Pillar 2: Perception (§4.3). *Input provenance tagging:* whether all inbound content is annotated with origin metadata (source identity, trust level, timestamp) before reaching the LLM context, corresponding to Layer P1 (Source Tagger). These tags persist through the pipeline and are consumed by downstream cognition and execution pillars. OpenFang earns ◦ because its `TaintSource/TaintLevel` labels attach origin information to messages within the runtime, but do not produce the full *ProvenanceTag* metadata (origin, timestamp, trust level, detection warnings) that AGENTKERNEL’s P1 generates; no other compared system tags inputs with provenance at the perception boundary. *Graduated input filtering:* a multi-layer perception pipeline (P1–P4) that applies increasingly expensive defenses in sequence. AGT earns ◦ because its policy engine evaluates declarative rules on inter-agent messages before they reach the agent, providing single-layer input checking but not a graduated multi-layer pipeline; OpenFang earns ◦ for its prompt-injection scanner, which operates as a single detection pass without layered depth.

Pillar 3: Cognition (§4.4). *Context & memory management:* structured lifecycle management of agent memory (archival, retrieval, scoping). Letta earns • for its self-managed context windows; AIOS earns ◦ for its `StorageManager/LSFS` mount-point-based file isolation; OpenFang earns ◦ for per-message state tracking implied by its taint-label infrastructure; SmythOS earns ◦ for its pluggable `Storage/VectorDB` connectors. *Memory taint tracking:* information-flow labels propagated at per-entry granularity. OpenFang earns ◦ for its `TaintSource/TaintLevel` labels, which operate within the runtime but do not propagate through a graduated perception pipeline. *Provenance-aware retrieval:* retrieval decisions conditioned on taint labels and provenance chains, not just semantic similarity. No compared system implements this.

Pillar 4: Execution (§4.5). *Tool & workflow orchestration:* managing tool routing, skill composition, and multi-step workflows. OpenFang and SmythOS earn • for rich plugin/connector ecosystems; AIOS earns ◦ for its `syscall`-style tool API. AGT’s policy engine evaluates governance rules on tool calls but does not perform tool routing or workflow composition, hence —. *Agent scheduling & concurrency:* concurrent agent execution with isolation. AIOS earns • for its FIFO/round-robin scheduler; OpenFang and SmythOS earn ◦ for limited concurrency support. *Syscall-level sandboxing:* OS-kernel-level process isolation (eBPF, Landlock, Seatbelt). nono earns • for Landlock/Seatbelt-based sandboxing; OpenFang earns ◦ for WASM metering sandboxes that do not reach the host kernel. *Semantic-to-syscall enforcement:* bridging high-level permission semantics to low-level kernel hooks (Layers E1–E3). No compared system links semantic policy to `syscall`-level enforcement. *Plan–trace alignment:* post-execution verification that compares declared plans to actual execution traces (Layer E4). OpenFang earns ◦ for its Merkle audit chains, which provide tamper-evident recording of actions but do not automatically align recorded traces against stated plans. AGT’s pre-execution policy checks and SLO monitoring are governance mechanisms, not post-hoc plan–trace comparison, hence —.

Cross-Pillar (Principle 3, §3). *Cross-lifecycle integration:* whether identity, perception, cognition, and execution share provenance labels and taint metadata across the full agent lifecycle. OpenFang earns ◦ because its security features (taint labels, audit chains, capability gating) compose within the runtime for

hosted agents but do not extend across externally connected agents. AGT’s components span multiple lifecycle phases but are independently adoptable modules that do not share unified provenance or taint metadata, hence —.

Scoring criteria. • indicates comprehensive, architecturally integrated support as a core design element; ◦ indicates partial or optional support (e.g., available as an add-on, limited to a subset of agents, or lacking cross-pillar integration); — indicates the capability is absent or not a design focus. Ratings reflect our reading of each project’s public documentation and open-source repositories as of May 2026; they are qualitative author assessments, not independently audited certifications.

6.1 The Agent Harness Landscape

6.1.1 Orchestration Frameworks

Orchestration frameworks (e.g., LangChain/LangGraph [12, 20], AutoGen [21], Microsoft Agent Framework [15], Semantic Kernel [17], CrewAI [16], MetaGPT [14], AutoGPT [11], and smolagents [50]) collectively represent widely adopted agent infrastructure, with large downstream ecosystems spanning vendors, cloud providers, and open-source communities. They manage agent workflows, tool routing, retries, and inter-agent messaging.

Distinction: Their primary contribution is building *stronger agents*—better reasoning chains, more reliable tool use, and smoother multi-agent coordination. Security ranges from absent to optional enterprise plugins (e.g., smolagents documents that its built-in LocalPythonExecutor is not a security sandbox and recommends external isolation for untrusted code [50]). AGENTKERNEL integrates with any orchestrator through three narrow adapter interfaces (Principle 5) and provides the OS layer that no orchestration framework offers. The relationship is complementary: orchestrators build capable agents; AGENTKERNEL provides the OS those agents run on.

6.1.2 Agent Runtimes

A growing class of projects positions itself between low-level orchestration frameworks and full governance platforms by providing *runtime infrastructure services* for AI agents, often borrowing OS terminology such as “kernel,” “scheduling,” and “memory management.” We call these *agent runtimes* and note that the term covers a wide spectrum of deployment models, abstraction levels, and security ambitions. To disambiguate this spectrum—and to clarify how it relates to AGENTKERNEL—we first characterize four representative systems from their source code and public documentation, then summarize the structural differences in Table 4.

What agent runtimes provide. Despite the shared “OS” framing, agent runtimes differ sharply in *what* they abstract and *how* they enforce:

- **AIOS** [22] (and the conceptual “LLM as OS” paper [51]) ships a user-space “kernel” as a *FastAPI/uvicorn HTTP server*: the Cerebrum agent SDK talks to it via `POST /query` over localhost, while agents submitted through `/agents/submit` run inside the same server process in a thread pool. The kernel wraps a request queue and FIFO/round-robin scheduler for concurrent agents; a `StorageManager` (LSFS) offers mount-point-based file isolation; and a `syscall`-style API (`llm_syscall`, `memory_syscall`, `tool_syscall`) defines the interaction surface. Despite the server boundary, the separation is an *API convention*: agents co-hosted in the server share the same Python address space and can bypass the `syscall` layer directly, while remotely connected agents gain no stronger isolation than an unauthenticated HTTP endpoint (the API has no authentication middleware). Access control exists as a stub `PermissionManager` without cryptographic identity or taint labels.

Table 4: Agent runtime deployment models and enforcement characteristics. Deployment indicates whether the runtime runs in the same process as the agent (**in-proc**), as a separate service (**service**), or in a hybrid mode. Security enforcement indicates whether security mechanisms are mandatory or opt-in. AGENTKERNEL is included for comparison; its deployment form is explicitly secondary to the non-bypass invariant (Principle 5).

System	Deployment	Primary focus	Security
AIOS	HTTP server + in-proc agents	LLM scheduling	No auth; stub ACL
OpenFang	Daemon / in-proc kernel	Skill/Plugin sandbox	Mandatory for hosted; opt-in for external
SmythOS	In-proc library (npm)	Unified resources	ACL + SES sandbox
Letta	Standalone server	Memory management	Optional sandbox
AGENTKERNEL	Form-agnostic	Security kernel	Mandatory, non-bypassable

- **OpenFang** [23] compiles to a *single Rust binary*: `openfang start` runs `OpenFangKernel` plus an Axum HTTP API in one daemon (default `127.0.0.1:4200`), while CLI commands without a daemon boot the same kernel in-process. External clients reach agents via REST/WebSocket/SSE; kernel-hosted agents execute *inside* the daemon (LLM loops as async tasks; optional WASM or Python modules) and invoke kernel services through an in-process `KernelHandle` callback rather than HTTP. Its security feature set is the richest among the runtimes: WASM dual-metering sandboxes (wasmtime with instruction-count and wall-clock epoch limits), Ed25519-signed agent manifests, Merkle audit chains, information-flow taint labels (`TaintSource/TaintLevel`), SSRF host allowlists, prompt-injection scanning, capability-gated tool access with parent→child inheritance validation, and MCP/A2A protocol adapters. For these hosted agents, mediation is mandatory—every inbound message traverses quota checks, capability filtering, and audit logging before the agent loop, and child agents cannot exceed their parent’s capability set. For *externally connected* agents (e.g., a LangChain A2A service), the runtime is only an integration endpoint: A2A/MCP forwarding does not bring the peer’s LLM calls, memory, or tool use inside the kernel perimeter.
- **SmythOS SRE** [24] brands itself as “The Linux of AI Agents.” Its open-source Smyth Runtime Environment is an *in-process Node.js library*: a singleton `SRE.init()` bootstraps 16 subsystem services (LLM, Storage, VectorDB, Vault, Scheduler, Telemetry, etc.) inside the host application’s process. The “OS” metaphor materializes as a *Connector pattern*: pluggable backends (local/S3, RAM/Redis, Pinecone, etc.) expose a unified API so that `storage.write()` is backend-agnostic. Each agent receives a scoped resource view via an `AccessCandidate` identity (`User/Agent/Team`), with decorator-enforced ACLs (`Read/Write/Owner`) and SES-based JavaScript sandboxing for user-supplied code. However, the runtime carries no cryptographic workload identity, taint labels, or kernel-level enforcement boundary; the commercial SmythOS cloud platform (separate from this open-source runtime) adds hosted infrastructure but not OS-kernel-class security.
- **Letta** [25] (formerly MemGPT) runs as a *standalone FastAPI server* offering “agents as a service”: agents are persistent, stateful objects whose core, archival, and recall memory is managed by the server and accessed through REST/SDK. Its core innovation is *self-managed context windows*: agents use built-in tools to archive, retrieve, and edit their own memory blocks. Tool sandboxing exists as a three-mode option (`local / e2b / local_sandbox`), defaulting to in-process execution with no isolation. No information-flow labels, provenance tags, or cryptographic identities are applied to memory items.

Other works in this space include KAOS [52], which builds a multi-agent OS on Kylin Linux; AutoAgent [53], which frames itself as an OS enabling zero-code agent creation; and the Agent-Centric OS review [54] and AI-in-OS survey [55], which provide conceptual roadmaps.

How runtimes relate to agents and to AGENTKERNEL. Agent runtimes span a spectrum between “part of the agent” and “mandatory management layer,” and the distinction matters for security. An in-process runtime like SmythOS SRE shares the agent’s address space; its ACL decorators can be bypassed by any code that reaches the underlying connector object. AIOS runs agents in the same server process via a thread pool, so co-hosted agents can access kernel internals directly despite the HTTP syscall facade; remotely connected agents gain no stronger isolation than an unauthenticated HTTP endpoint. Even OpenFang—which enforces mandatory capability checks and audit logging for agents *hosted inside* its kernel—cannot extend those guarantees to externally connected A2A agents whose LLM calls, memory accesses, and tool invocations remain outside the kernel’s perimeter. Letta’s server-mode boundary isolates agent state but carries no information-flow constraints on memory items.

AGENTKERNEL differs on a single architectural invariant: the three adapter interfaces (LLM, tool, storage) constitute the *only* path to the capabilities those adapters guard (Principle 5). Whether AGENTKERNEL is deployed as an embedded SDK, a gateway, or a sidecar is a deployment decision; the non-bypass guarantee is preserved regardless, because ambient credentials and direct transports are not available to the agent.

The deployment similarities between service-mode runtimes and a sidecar AGENTKERNEL are architectural surface only. A runtime provides *opt-in platform services* that agents may or may not use; AGENTKERNEL provides a *mandatory mediation layer* that agents cannot circumvent. Even OpenFang’s security modules—while individually sound—do not compose into a unified lifecycle discipline: taint labels are tracked within the runtime but do not propagate through a graduated perception pipeline or govern long-term memory retrieval, and the enforcement scope is bounded to the OpenFang-hosted subset of the agent ecosystem.

Distinction: Agent runtimes advance *agent capability and developer experience*: better scheduling (AIOS), richer memory (Letta), broader integrations (SmythOS), and execution isolation (OpenFang). AGENTKERNEL advances *mandatory, lifecycle-spanning security*: it provides the non-bypassable OS kernel that these runtimes—or the orchestrators and governance platforms above and below them—can adopt to turn their opt-in services into externally auditable guarantees.

6.1.3 Governance Platforms

Unlike agent runtimes, which provide the execution substrate itself, *governance platforms* sit alongside agent code and supply observability, policy enforcement, and compliance reporting. Critically, these platforms integrate via **code-level SDKs**—imported as libraries or decorators into the agent’s own process—rather than as external, non-bypassable enforcement boundaries. This design gives governance platforms broad framework compatibility but limits their security guarantees: any code running in the same process can, in principle, bypass the governance hooks.

Observability and evaluation stacks. AgentOps [28] and LangSmith [27] provide Python/TypeScript SDKs that instrument agent code through decorators (@traceable), client wrappers (wrap_openai, wrap_anthropic), and ASGI middleware, forwarding trace spans, evaluation metrics, and run logs to a hosted analytics backend. LangSmith additionally ships an experimental cloud-hosted sandbox module (Docker-based isolated containers accessed through a SandboxClient API) for executing untrusted code in a remote environment—architecturally similar to E2B rather than to an in-process policy engine. Despite this execution capability, the governance relationship remains *opt-in*: agents may bypass the instrumented path or invoke tools outside the sandbox, because no mandatory interposition prevents direct access to LLMs, local tools, or memory.

Policy-enforcement toolkits. Microsoft’s Agent Governance Toolkit (AGT) [26] is the most comprehensive industry effort. It ships as a set of *application-level Python packages* that run *inside the same process* as the agent application. Its components include:

- **Agent OS** (confusingly named—an in-process deterministic policy engine, not an OS kernel): evaluates declarative policies before each tool call or inter-agent message;
- **AgentMesh** [56]: a sidecar-deployable zero-trust layer providing SPIFFE/SVID workload identity and protocol bridges (A2A, MCP, IATP);
- **Agent Hypervisor**: execution rings with resource quotas and circuit-breaker semantics;
- **Agent SRE**: SLO monitoring, chaos injection, and automated remediation.

Because the core policy engine is an in-process library, agents and the governance logic share a single trust boundary. A compromised agent—or any code with access to the interpreter—can mutate policy state, skip evaluation hooks, or invoke tools directly. AgentMesh provides genuine cryptographic workload identity via out-of-process SPIFFE/SVID attestation (hence • for that row in Table 3) and enables cross-org attestation through SPIFFE federation (◦). However, SPIFFE binds the runtime workload but does not trace provenance across developer, code artifact, and operator simultaneously (hence — for multi-dimensional provenance), and tool execution and memory access still traverse no mandatory mediation.

Other work explores agent identity management [34], zero-trust authentication frameworks [57], and cryptographic human-delegation provenance [58].

Distinction: AGT and AGENTKERNEL share the *goal* of governing agent actions but differ on a fundamental architectural choice: *where* the governance logic executes relative to the agent’s trust boundary. AGT interposes governance as a co-located library—modular, independently adoptable, and compatible with any Python/TS/.NET/Rust/Go agent [26]—but cannot guarantee non-bypassability because governance and agent share an address space. AGENTKERNEL places governance in a *separate kernel* that controls the *only* interfaces through which agents reach LLMs, tools, and storage (Principle 5). Identity, perception, cognition, and execution share unified taint metadata enforced below the agent’s address space (e.g., via eBPF-class hooks), making bypass equivalent to escaping the OS kernel. Where AgentMesh adds SPIFFE/SVID identity as an adjacent sidecar, AGENTKERNEL folds mutual attestation, verifiable credentials, and delegation semantics into the same kernel abstraction that mediates tool and storage access—eliminating the gap between “who you are” and “what you may do.” The relationship is thus complementary at different trust levels: AGT’s rich policy language and framework adapters can run *above* AGENTKERNEL, gaining non-bypassable enforcement from the kernel below without sacrificing their application-level expressiveness.

6.1.4 Execution Sandboxes

Execution sandboxes isolate agent execution at the OS or cloud level. nono [29] offers Landlock/Seatbelt-based kernel sandboxing, Sigstore-based signing of agent instruction bundles, credential proxy injection, and network allowlists. E2B [30] provides cloud-side isolated environments with Python/JS SDKs. Anthropic’s sandbox-runtime [31] enforces filesystem and network restrictions at the OS level without a container runtime.

Distinction: These tools provide effective *execution-layer isolation*. They do not, by themselves, subsume AGENTKERNEL’s full-stack treatment: even where a sandbox adds supply-chain controls (e.g., Sigstore attestation of skill instruction files in nono [29]), it does not replace graduated perception filtering and memory taint propagation across the entire agent pipeline. Their isolation is primarily binary (inside vs. outside the sandbox), whereas AGENTKERNEL provides *graduated, context-aware* enforcement. Each tool invocation receives a dynamically computed allowlist derived from the agent’s identity, the provenance of

its inputs, and the taint level of the memory entries. AGENTKERNEL’s execution pillar subsumes sandbox functionality and extends it with semantic context.

6.2 The Missing OS Layer: Why an OS, not just a stronger harness?

The pattern across these four tiers is revealing. Each tier excels within its scope, but no tier provides a *unified* OS layer with identity, perception, cognition, and execution as integrated services sharing provenance labels, trust signals, and taint metadata. The gap is not a missing feature; it is a missing *architectural layer*.

The distinction between an agent harness and an agent OS parallels the distinction between a user-space library and an OS kernel. A library provides services to applications that *choose* to call it; a kernel provides mandatory services that *all* applications must traverse. Current harness solutions operate as opt-in libraries: an agent framework can skip the guardrail, bypass the policy engine, or ignore the sandbox. AGENTKERNEL operates as a mandatory kernel: because it controls the *only* interfaces through which agents can reach LLMs, tools, and storage (Principle 5), a compromised or misconfigured agent cannot bypass the OS layer any more than a compromised user-space process can bypass the OS kernel.

This mandatory mediation simultaneously improves both security and capability. Security improves because enforcement is non-bypassable. Capability improves because operators can *trust* the enforcement boundary, enabling them to grant agents broader tool access, richer memory sharing, and deeper delegation chains—all of which would be too risky without OS-level guarantees.

6.3 AGENTKERNEL as the Agent OS Kernel

AGENTKERNEL provides the *operating system kernel* that transforms a collection of harness components into an integrated, trustworthy agent platform. The kernel consists of three components: the *Global Agent Registry* (the remote trust anchor), the *Agent Kernel* (the local trust anchor holding the private key), and the *three adapter interfaces* (LLM adapter, tool adapter, storage adapter) through which all capability interactions must pass. Everything else lies *outside* the kernel and is assumed potentially compromised.

The kernel contributes three services that no orchestration or governance layer alone provides:

1. **Identity infrastructure:** Mutual attestation, verifiable AICs, and capability-bounded delegations.
2. **Information-flow control across the pipeline:** Provenance labels flow from perception through cognition to execution.
3. **Execution governance with kernel-level enforcement:** Tool executions exceeding declared permissions are blocked at the OS level, not just logged.

AGENTKERNEL inverts the relationship where infrastructure logic is entangled with agent business logic. The OS kernel is a *standalone component* that can integrate as an *embedded SDK*, *middleware*, or a *sidecar*. The agent framework can be swapped without re-implementing OS services.

6.4 Implications for A2A and H2A Interactions

Agent-to-Agent (A2A). AGENTKERNEL’s identity pillar provides the necessary trust infrastructure for agents from different organizations to interoperate [43, 59]. Mutual attestation establishes identities, and session tokens scope the interaction.

Human-to-Agent (H2A). Users need assurance that the agent acted as instructed. AGENTKERNEL’s execution pillar provides this through plan–trace alignment and immutable audit logs, while the OIDC-bound operator identity ensures accountability.

6.5 Other Related Areas

GUI and computer-use agents. A vibrant line of work builds agents that interact through graphical user interfaces [9, 60, 19, 18, 61, 62], with comprehensive surveys [63, 64] and safety benchmarks including OSWorld [65], OS-Harm [33], and PASB [6]. These focus on *interaction modality* and operate *within* the OS as applications; AGENTKERNEL operates *below* such agents as infrastructure and can serve as the enforcement layer.

Agent memory systems. MemOS [36] and MemoryOS [37] optimize memory *capacity and retrieval quality*; their published designs do not center on memory *security* in the sense of information-flow labels or taint propagation across agent subsystems. AGENTKERNEL’s cognition pillar supplies that missing enforcement-oriented layer.

Multi-agent platforms. Platforms like OpenHands [13], SWE-agent [66], Magentic-One [10], and AgentStore [67] are engineering platforms where security is optional; AGENTKERNEL provides the OS kernel that such platforms can adopt.

Agent protocols. Protocols like MCP [68], A2A [59], ANP [69], Agent-OSI [70], and BlockA2A [43] specify *what agents can say*; AGENTKERNEL ensures *what agents can do*—the two are complementary.

7 Discussion and Future Directions

Beyond the design and analysis presented above, we summarize several architectural insights, examine their implications for how the field thinks about agent security, and outline the open problems and research directions they motivate.

7.1 Architectural Insights

The agent is outside the TCB. A conventional reading of agent security treats the agent—its reasoning loop, its tools, its memory—as the entity to be *protected*. AGENTKERNEL’s architecture inverts this: the agent (LLM, application logic, orchestration fabric) is treated as an *untrusted compute substrate* that the security kernel *contains*. The kernel mediates every interaction—inbound perception, memory storage and retrieval, tool invocation—and the agent cannot reach any resource without kernel approval. This inversion generalizes beyond AGENTKERNEL: it implies that agent security architectures should not ask “how do we protect the agent?” but “which components must we trust to constrain an agent that may behave adversarially?” Concretely, the trusted computing base consists of the GAR, the Agent Kernel, the three adapters, the eBPF subsystem, and the cryptographic primitives (§5.1). The agent, its LLM, and its application code are explicitly excluded.

Provenance as the security nervous system. A second insight is that provenance is not merely a feature of one pillar but the *coupling substrate* that transforms four independent enforcement mechanisms into a single coherent security argument. Identity establishes *who* produced content; Perception assigns initial trust levels; Cognition propagates taint labels through memory derivations; Execution consumes these labels to build allowlists and constrain tool actions. This provenance flow—labeled, auditable, and mandatory—is the agent-level analogue of labeling in classical mandatory access control (Bell–LaPadula, Biba) [48, 49], but applied to the semantic plane rather than to files and processes. The architectural lesson is that provenance must be a *first-class OS primitive* in agent systems, not an application-level annotation that agents may or may not maintain.

The reference monitor at the semantic plane. Classical OS kernels mediate physical resources: memory pages, file descriptors, process address spaces. AGENTKERNEL demonstrates that the same architectural pattern—a non-bypassable, tamper-proof, always-on authority that mediates every resource transition [40]—applies to *semantic resources*: identity claims, perception labels, memory taint, and execution intent. This suggests that the reference monitor concept is more general than its conventional instantiation in OS kernels: when the resources to be mediated are semantic (natural language, memory entries, tool plans), the reference monitor operates at the semantic level while anchoring enforcement to the physical syscall boundary. The semantic–syscall bridge (Property 5.2) is the architectural mechanism that realizes this dual-level mediation.

Scope of the lifecycle decomposition. The four-pillar decomposition follows from the perception–cognition–execution lifecycle with identity cutting across all phases (§2.1). This decomposition is appropriate for *interactive, tool-augmented, memory-bearing* agents—the dominant paradigm in current practice. Purely reactive agents, agent swarms with emergent rather than hierarchical delegation, and agents that do not maintain persistent memory may not require all four pillars. In such settings, AGENTKERNEL degrades gracefully: the identity pillar operates independently; the perception and execution pillars remain mandatory for any agent that ingests external content or invokes tools; only the cognition pillar’s information-flow guarantees depend on the presence of durable memory. Characterizing the minimal pillar configuration for a given agent class is an open question.

7.2 Security as a Capability Multiplier

A recurrent concern with security-first designs is that security constrains what agents can do. AGENTKERNEL’s architecture demonstrates the opposite: each security pillar simultaneously *enables* agent capabilities that are unattainable without OS-level enforcement. This is not a coincidental property but a structural consequence of non-bypassable mediation.

The underlying principle can be stated precisely: in any agent system with non-bypassable security mediation, the set of capabilities that can be *safely delegated* to the agent grows monotonically with the strength of the security invariants. The reason is that non-bypassable enforcement transforms the operator’s question from “can the agent be trusted with action X?” to “can the agent be *prevented* from exceeding the permitted scope of X?” —a fundamentally easier question because the answer depends on the correctness of the enforcement layer, not on the agent’s future behavior.

This logic plays out concretely across the four pillars:

- **Identity** → **cross-organizational A2A**. Without cryptographically verified identity and kernel-mediated mutual attestation, agent-to-agent collaboration is confined to agents within a single trust domain, because verifying the counterparty’s identity, authorization, and delegation chain is infeasible at the application layer. AGENTKERNEL’s identity pillar—where K_{priv} never leaves the kernel boundary and the GAR serves as the root of trust—unlocks marketplace-style agent discovery, cross-organization delegation, and accountable multi-hop workflows.
- **Perception** → **richer input channels**. When all external content enters the LLM context without mandatory mediation, operators rationally restrict input modalities as a precaution—disabling browser access, stripping multimodal content, or requiring human review at every turn. The graduated perception pipeline (P1–P4) provides defense-in-depth with provenance-tagged outputs, enabling agents to safely ingest richer and higher-entropy input sources.
- **Cognition** → **trustworthy long-horizon memory**. Without provenance labels, retrieval from long-term memory degrades as stores grow: all entries appear equally trustworthy, and a single poisoned entry can silently influence decisions across sessions. AGENTKERNEL’s item-level taint propagation enables provenance-weighted retrieval, so high-confidence entries are preferentially surfaced and contaminated

entries carry explicit warnings—turning memory from a liability into a reliable asset for long-horizon reasoning.

- **Execution** → **expanded tool access with non-bypassable guardrails.** Application-layer guardrails can inspect tool *parameters* but cannot constrain tool *execution*: a malicious or buggy tool can spawn child processes, access undeclared files, or open network connections. The semantic-syscall bridge (E1–E3) translates declared permissions into dynamically installed eBPF allowlists before the tool’s process tree is spawned, giving operators confidence to expand tool sets and grant write access that would be too risky with application-layer-only enforcement.

The net effect is that AGENTKERNEL does not trade security against capability. Security is the *mechanism* through which capability is safely expanded: stronger invariants at each pillar translate directly into broader, safer delegation. Returning to the motivating DevOps scenario (§1), each failure in the chain—impersonation, injection, memory poisoning, over-privileged execution—corresponds to an absent invariant; supplying that invariant simultaneously closes the security gap and removes the corresponding capability ceiling.

7.3 Integration with the Agent Ecosystem

AGENTKERNEL is designed as a complementary OS layer, not a replacement for existing agent infrastructure. This design choice reflects a classical systems principle: different layers of the stack optimize different concerns, and clean interfaces enable independent evolution. Just as TCP/IP did not replace Ethernet and the web did not replace TCP/IP, AGENTKERNEL adds a *mandatory security mediation layer* beneath orchestration frameworks, agent runtimes, governance platforms, and execution sandboxes—each of which continues to optimize its own concern (workflow composition, scheduling, policy authoring, process isolation).

The three adapter interfaces (LLM, tool, storage) define the integration boundary. Any orchestration framework or agent runtime can adopt AGENTKERNEL by routing its model calls, tool invocations, and memory operations through these adapters, gaining identity, perception, and taint-aware memory without re-architecting its scheduling or workflow logic:

- **Beneath agent runtimes.** AIOS [22] or OpenFang [23] can retain their scheduler and resource manager while AGENTKERNEL provides the security substrate: identity-driven workload isolation, perception mediation for all LLM-bound content, and provenance-aware memory for cross-session state.
- **Beneath governance platforms.** Microsoft’s Agent Governance Toolkit (AGT) [26] provides policy authoring and audit pipelines; AGENTKERNEL provides the non-bypassable enforcement substrate beneath AGT’s policy engine, closing the gap between policy evaluation and kernel-level constraint installation.
- **Above execution sandboxes.** Execution sandboxes such as nono [29] and E2B [30] provide process isolation; AGENTKERNEL provides the semantic context—identity, taint, intent alignment—that determines *what* the sandbox should permit for a given invocation. The sandbox enforces isolation; AGENTKERNEL supplies the dynamically computed policy.

This layered model is not merely a pragmatic accommodation of existing ecosystem investments. It is an architectural commitment: the agent OS kernel should be *minimal*—it should mediate only what must be enforced below the agent’s control plane—and delegate to higher or lower layers for concerns best addressed there. The three adapters are the contract that bounds this minimality.

7.4 Limitations and Open Problems

Section 5.5 analyzed residual risks that fall outside AGENTKERNEL’s current security guarantees: taint fidelity under adversarial segmentation, semantic firewall accuracy, GAR centralization, the non-bypass assumption, eBPF portability, and deferred formal verification. Here we address broader architectural and systems challenges that constrain AGENTKERNEL’s deployability and motivate the research agenda in §7.5.

LLM-based components in the TCB. Perception Layer P3 (Semantic Firewall) and Execution Layer E2 (LLM Validator) are LLM-based classifiers that sit *inside* the trusted computing base: their decisions directly affect which content reaches the agent and which actions are permitted. The adversarial robustness of these components is not yet characterized for agent-security workloads, and the defense-in-depth argument (Principle 3) mitigates but does not eliminate the risk of a classifier-level failure cascading across pillars. Reducing reliance on LLM-based enforcement—or proving that the remaining deterministic layers (P1–P2, E1, E3) provide sufficient independent backstop—is a priority for strengthening the TCB.

TCB size and heterogeneity. The combined TCB (GAR, Agent Kernel, three adapters, eBPF subsystem, LLM classifiers, and cryptographic primitives) is large and spans multiple implementation domains (distributed services, kernel modules, language-model inference). While the adapters narrow the agent-visible attack surface (Principle 5), the TCB *itself* has not been minimized. A natural direction is to ask which components are strictly necessary: can the LLM Validator (E2) be eliminated if deterministic checks (E1) and eBPF enforcement (E3) are sufficiently expressive? Can the GAR be decentralized through threshold signatures or a transparency log, reducing the trust concentrated in a single CA?

Policy specification and human factors. AGENTKERNEL’s security guarantees assume correct policy—capability boundaries $S_{\max}$, tool manifests, taint lattice configurations—but the paper has not addressed the human problem of *specifying* these policies. Who writes an agent’s capability boundary? How are tool manifests validated against implementation behavior? When policies from different organizations conflict in a cross-GAR deployment, what is the conflict resolution procedure? Operator-facing tooling for policy authoring, simulation, and audit is essential for deployment feasibility and represents a significant systems and HCI research challenge.

Non-bypassability in heterogeneous deployments. The guarantees in §5.2 are predicated on Principle 5: the three adapters must be the *only* paths through which agents reach LLMs, tools, and storage. In heterogeneous production environments—where agents may have pre-existing ambient credentials, direct filesystem access, or side channels through orchestration infrastructure—ensuring this property is an operational challenge, not a theoretical one. Automated verification of deployment integrity (e.g., checking that no file descriptors or network sockets bypass the adapters) would strengthen the deployment-time security argument.

Empirical characterization. While the architecture targets low enforcement overhead, comprehensive performance and security characterization under realistic workloads is pending. Three empirical questions are particularly salient: (i) the latency distribution of eBPF probe installation/teardown per tool invocation under concurrent agent workloads; (ii) the scalability of lattice-search operations over large memory stores with millions of provenance-tagged entries; and (iii) the end-to-end security posture under adversarial stress, including taint fidelity when source-segmentation boundaries are adversarially blurred (§5.5) and semantic firewall calibration across diverse application domains.

7.5 Research Agenda

The limitations above define a research agenda organized around four themes.

Formal foundations. Mechanized verification of the structural guarantees claimed in Sections 5.2 and 5.4.1 would elevate them from architectural claims to machine-checked theorems. Priority targets include: the monotonic attenuation chain (Property 5.1), verified via Coq or Lean against the AIC issuance and kernel-mediated delegation protocols; the policy intersection semantics (Principle 2), formalized as a lattice of capability sets with intersection as the composition operator; and TLA+ models of the mutual attestation and delegation protocols (§4.2), covering liveness under concurrent delegation and safety under GAR compromise.

Formalizing the semantic–syscall bridge (Property 5.2) would connect these semantic-level models to the operational semantics of eBPF enforcement.

Empirical security evaluation. The companion empirical program should go beyond performance benchmarking to include adversarial evaluation of each pillar and of cross-pillar composition. For Perception: measuring false-positive/false-negative rates of the graduated pipeline against established prompt-injection and jailbreak benchmarks (OS-Harm [33], PASB, MobileSafetyBench). For Cognition: quantifying taint fidelity degradation under adversarially crafted content designed to blur source-segment boundaries. For Execution: evaluating whether plan–trace alignment (E4) reliably surfaces hallucinated or unauthorized actions, and whether the eBPF allowlist construction (E3) introduces race conditions under concurrent tool invocations. For composition: red-team exercises that attempt to exploit cross-pillar coupling (e.g., crafting a payload that simultaneously evades perception and induces incorrect taint labels that propagate to permissive execution allowlists).

Deployment and trust infrastructure. Several infrastructure challenges must be addressed for production deployment. *Kernel integrity assurance:* chaining the Agent Kernel’s trust anchor to a hardware root of trust via platform secure/measured boot, signed kernel images, and runtime integrity monitoring, so that remote peers can verify—before mutual attestation—that the mediation layer itself has not been substituted or downgraded. *Cross-GAR federation:* as multiple GARs emerge in a multi-stakeholder ecosystem, mutual recognition protocols analogous to cross-certification in PKI are needed, together with formal models of trust propagation across GAR boundaries. *Multi-agent security composition:* when multiple AGENTKERNEL-protected agents interact, how do their security invariants compose? The intersection semantics (Principle 2) extend naturally to pairwise A2A sessions, but multi-party composition (e.g., a workflow spanning agents governed by different GARs with incompatible taint lattices) raises open questions about invariant preservation under heterogeneous trust models.

Ecosystem standards and advanced primitives. The agent ecosystem needs shared abstractions—analogue to POSIX for traditional OS services—for identity, memory provenance, and execution governance. Building on W3C Verifiable Credentials [42] with agent-specific semantics (capability boundaries, delegation chains, revocation semantics) and standardizing the adapter interfaces would enable interoperability across agent runtimes, governance platforms, and execution sandboxes without fragmenting the security model. On the cryptographic front, three extensions are natural: *privacy-preserving attestation* via zero-knowledge proofs, enabling an agent to prove it possesses a required capability without revealing its full identity; *post-quantum readiness* by supporting PQ signature algorithms (e.g., ML-DSA, SLH-DSA) as drop-in replacements for Ed25519 in the AIC and attestation protocols; and *TEE-backed key custody* extending the tiered custody path (§4.2) with formal verification of the enclave interface.

These four themes are interdependent: formal models inform what should be empirically tested; deployment experience reveals which formal guarantees matter most; standards encode what the ecosystem converges on. Together they define the path from a positioning paper to a deployed agent OS security layer. Section 8 summarizes our contributions toward this agenda.

8 Conclusion

Agents routinely cross trust boundaries in a single session; today’s orchestration frameworks, agent runtimes, governance platforms, and execution sandboxes each cover part of the problem, but none supplies a *unified, mandatory* layer that mediates identity, perception, memory, and execution end-to-end. We argued that agents need an operating system, not only a stronger harness, and that security should be the organizing principle of that layer.

AGENTKERNEL is such an agent OS. Four pillars—Identity, Perception, Cognition, and Execution—enforce mediation at every boundary the agent crosses, grounded in classical reference-monitor and information-flow ideas and extended to semantic threats such as prompt injection, memory poisoning, and unsafe tool use. A unifying theme runs through the design: security, when made structural rather than policy-based, simultaneously enables richer agent capabilities. Kernel-managed identity supports trustworthy collaboration across agents and organizations; graduated perception lets agents safely ingest richer input channels; taint-aware memory separates reliable knowledge from contaminated content; and kernel-backed execution mediation lets operators widen tool access without abandoning non-bypassable guarantees. AGENTKERNEL sits beneath the broader harness ecosystem as a complementary OS layer rather than a replacement for orchestration frameworks, agent runtimes, governance platforms, or execution sandboxes.

As agents take on higher-stakes tasks, the same class of services that traditional kernels provided for applications—strong identity, mediated I/O, governed memory, and controlled execution—will be indispensable at the semantic level. We present AGENTKERNEL as a concrete step toward that foundation and hope it informs further work on agent-native operating systems.

References

- [1] Anywhere. Cursor: AI-native editor, tab completion, and coding agents. <https://cursor.com/>, 2026. Commercial product; accessed May 2026.
- [2] Anthropic. Claude Code: Agentic coding tool for terminal and IDE. <https://claude.com/product/claude-code>, 2026. Commercial product; accessed May 2026.
- [3] GitHub. GitHub Copilot documentation. <https://docs.github.com/copilot>, 2026. Official documentation hub; accessed May 2026.
- [4] GitHub. Openai codex coding agent in GitHub Copilot. <https://docs.github.com/en/copilot/concepts/agents/openai-codex>, 2026. Coding agent documented under Copilot; accessed May 2026.
- [5] OpenClaw. OpenClaw: Personal AI assistant framework. <https://github.com/openclaw/openclaw>, 2026. Open-source repository; accessed May 2026.
- [6] Yuhang Wang, Feiming Xu, Zheng Lin, Guangyu He, et al. From assistant to double agent: Formalizing and benchmarking attacks on OpenClaw for personalized local AI agent. *arXiv preprint arXiv:2602.08412*, 2026.
- [7] Nous Research. Hermes Agent: Open-source agent harness with autonomous skills and scheduling. <https://github.com/NousResearch/hermes-agent>, 2026. Accessed May 2026.
- [8] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.
- [9] Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. OS-Copilot: Towards generalist computer agents with self-improvement. *arXiv preprint arXiv:2402.07456*, 2024.
- [10] Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, et al. Magentic-one: A generalist multi-agent system for solving complex tasks. *arXiv preprint arXiv:2411.04468*, 2024.
- [11] Significant Gravitas. AutoGPT: The vision of accessible AI for everyone. <https://github.com/Significant-Gravitas/AutoGPT>, 2025. 183 K GitHub stars as of Apr 2026.

- [12] LangChain, Inc. LangChain: The agent engineering platform. <https://github.com/langchain-ai/langchain>, 2025. 137 K GitHub stars as of May 2026.
- [13] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.
- [14] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2024.
- [15] Microsoft. Microsoft agent framework. <https://github.com/microsoft/agent-framework>, 2026. 10.5 K GitHub stars as of May 2026; successor to AutoGen/Semantic Kernel; Python/.NET multi-agent SDK.
- [16] CrewAI, Inc. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. <https://github.com/crewAIInc/crewAI>, 2025. 51 K GitHub stars as of May 2026.
- [17] Microsoft. Semantic Kernel: An SDK for integrating LLM technology into apps. <https://github.com/microsoft/semantic-kernel>, 2025. 28 K GitHub stars as of Apr 2026.
- [18] Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, et al. Mobile-agent-v3: Fundamental agents for GUI automation. *arXiv preprint arXiv:2508.15144*, 2025.
- [19] Chaoyun Zhang, Liqun Li, Shilin He, Xu Zhang, Bo Qiao, et al. UFO: A UI-focused agent for Windows OS interaction. In *Proceedings of NAACL*, 2025.
- [20] LangChain, Inc. LangGraph: Build resilient language agents as graphs. <https://github.com/langchain-ai/langgraph>, 2025. 32 K GitHub stars as of May 2026.
- [21] Microsoft. AutoGen: A programming framework for agentic AI. <https://github.com/microsoft/autogen>, 2024. 57 K GitHub stars as of Apr 2026; README states maintenance mode (no new features) with migration to Microsoft Agent Framework; see repository <https://github.com/microsoft/autogen> and <https://learn.microsoft.com/en-us/agent-framework/migration-guide/from-autogen/>.
- [22] Kai Mei, Xi Zhu, Wujiang Xu, Wenyue Hua, Mingyu Jin, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. AIOS: LLM agent operating system. *arXiv preprint arXiv:2403.16971*, 2024.
- [23] RightNow AI. OpenFang: Open-source agent operating system. <https://github.com/RightNow-AI/openfang>, 2026. 17.5 K GitHub stars as of May 2026; Rust; WASM sandbox, Merkle audit chain, Ed25519 signed manifests, prompt-injection scanning, MCP/A2A support.
- [24] SmythOS. SmythOS SRE: Cloud-native runtime for agentic AI. <https://github.com/SmythOS/sre>, 2026. 1.3 K GitHub stars as of May 2026; TypeScript; kernel + SDK + CLI monorepo with unified resource abstractions and ACL-based access control.
- [25] Letta AI. Letta: The platform for building stateful agents with advanced memory. <https://github.com/letta-ai/letta>, 2026. 22.7 K GitHub stars as of May 2026; formerly MemGPT; layered memory management and context control.
- [26] Microsoft. Agent governance toolkit: Runtime governance for AI agents. <https://github.com/microsoft/agent-governance-toolkit>, 2026. Modular platform comprising Agent OS (policy engine), AgentMesh (zero-trust SPIFFE/SVID identity), Agent Hypervisor (execution rings), Agent SRE, compliance verification, and shadow-AI discovery; 13 000+ tests; Python / TypeScript / .NET / Rust / Go SDKs; 1.6 K GitHub stars as of May 2026.

- [27] LangChain, Inc. LangSmith: AI agent & LLM observability platform. <https://www.langchain.com/langsmith/>, 2025. Framework-agnostic tracing, evaluation, and monitoring for LLM applications and AI agents.
- [28] Liming Dong, Qinghua Lu, and Liming Zhu. AgentOps: Enabling observability of LLM agents. *arXiv preprint arXiv:2411.05285*, 2024.
- [29] always-further. nono: A capability-based sandbox for AI agents. <https://github.com/always-further/nono>, 2026. 2.4 K GitHub stars as of May 2026; Landlock/Seatbelt kernel sandboxing, Sigstore signing of instruction files, credential proxy injection, network allowlists.
- [30] E2B. E2B: Open-source secure environments for AI agents. <https://github.com/e2b-dev/E2B>, 2026. 12.2 K GitHub stars as of May 2026; cloud-side isolated sandboxes for AI-generated code execution.
- [31] Anthropic. Sandbox runtime: Lightweight OS-level sandboxing without containers. <https://github.com/anthropic-experimental/sandbox-runtime>, 2026. 4.1 K GitHub stars as of May 2026; process-level filesystem and network restrictions at the OS level.
- [32] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pages 79–90, 2023.
- [33] Thomas Kuntz, Agatha Duzan, Hao Zhao, Francesco Croce, Zico Kolter, Nicolas Flammarion, and Maksym Andriushchenko. OS-Harm: A benchmark for measuring safety of computer use agents. *arXiv preprint arXiv:2506.14866*, 2025.
- [34] Tobin South, Subramanya Nagabhushanaradhya, et al. Identity management for agentic AI: The new frontier of authorization, authentication, and security for an AI agent world, 2025.
- [35] Yingxuan Yang, Huacan Chai, Yuanyi Song, Siyuan Qi, Muning Wen, et al. A survey of AI agent protocols. *arXiv preprint arXiv:2504.16736*, 2025.
- [36] Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, et al. MemOS: A memory OS for AI system. *arXiv preprint arXiv:2507.03724*, 2025.
- [37] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. Memory OS of AI agent. In *Proceedings of EMNLP*, pages 25961–25970, 2025.
- [38] Xinming Wei, Jiahao Zhang, Haoran Li, Jiayu Chen, et al. Agent.xpu: Efficient scheduling of agentic LLM workloads on heterogeneous SoC. *arXiv preprint arXiv:2506.24045*, 2025.
- [39] Abhinaba Basu. Tool receipts, not zero-knowledge proofs: Practical hallucination detection for AI agents. *arXiv preprint arXiv:2603.10060*, 2026.
- [40] James P. Anderson. Computer security technology planning study. In *Technical Report ESD-TR-73-51, USAF Electronic Systems Division*, 1972.
- [41] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.
- [42] Manu Sporny et al. Verifiable credentials data model v2.0. W3c recommendation, W3C, 2025.

- [43] Zhenhua Zou, Zhuotao Liu, Lepeng Zhao, and Qiuyang Zhan. BlockA2A: Towards secure and verifiable agent-to-agent interoperability, 2025.
- [44] Victor Costan and Srinivas Devadas. Intel SGX explained. Cryptology ePrint Archive, Paper 2016/086, 2016.
- [45] Bernard Ngabonziza, Daniel Martin, Anna Bailey, Haehyun Cho, and Sarah Martin. TrustZone explained: Architectural features and use cases. In *IEEE CIC*, pages 445–451, 2016.
- [46] Xinyue Shen, Zeyuan Chen, Michael Backes, Yun Shen, and Yang Zhang. “do anything now”: Characterizing and evaluating in-the-wild jailbreak prompts on large language models. In *Proceedings of ACM CCS*, pages 1671–1685, 2024.
- [47] Dorothy E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976.
- [48] David Elliott Bell and Leonard J. LaPadula. Secure computer systems: Mathematical foundations. *MITRE Technical Report*, (MTR-2547), 1973.
- [49] Kenneth J. Biba. Integrity considerations for secure computer systems. *MITRE Technical Report*, (MTR-3153), 1977.
- [50] Hugging Face. smolagents: A barebones library for agents that think in code. <https://github.com/huggingface/smolagents>, 2025. 27.3 K GitHub stars as of May 2026.
- [51] Yingqiang Ge, Yujie Ren, Wenyue Hua, Shuyuan Xu, Juntao Tan, and Yongfeng Zhang. LLM as OS, agents as apps: Envisioning AIOS, agents and the AIOS-agent ecosystem. *arXiv preprint arXiv:2312.03815*, 2023.
- [52] Zhao Zhuo, Rongzhen Li, Kai Liu, Huhai Zou, et al. KAOS: Large model multi-agent operating system. In *CCIS (CCKS-IJCKG 2024)*. Springer, 2025.
- [53] Jiabin Tang, Tianyu Fan, and Chao Huang. AutoAgent: A fully-automated and zero-code framework for LLM agents. *arXiv preprint arXiv:2502.05957*, 2025.
- [54] Shian Jia, Xinbo Wang, Mingli Song, and Gang Chen. Agent centric operating system – a comprehensive review and outlook for operating system. *arXiv preprint arXiv:2411.17710*, 2024.
- [55] Yifan Zhang, Xinkui Zhao, Ziyang Li, Guanjie Cheng, Jianwei Yin, et al. Integrating artificial intelligence into operating systems: A survey on techniques, applications, and future directions. *arXiv preprint arXiv:2407.14567*, 2024.
- [56] Microsoft. AgentMesh: Production-grade trust layer for multi-agent systems. <https://github.com/microsoft/agent-governance-toolkit/tree/main/agent-governance-python/agent-mesh>, 2026. Part of the Agent Governance Toolkit; SPIFFE/SVID-based identity, policy engine, A2A/MCP/IATP protocol bridge.
- [57] Ken Huang, Vineeth Sai Narajala, John Yeoh, Jason Ross, et al. A novel zero-trust identity framework for agentic AI: Decentralized authentication and fine-grained access control. *arXiv preprint arXiv:2505.19301*, 2025.
- [58] Asiri Dalugoda. HDP: A lightweight cryptographic protocol for human delegation provenance in agentic AI systems. *arXiv preprint arXiv:2604.04522*, 2026.

- [59] Google. Agent-to-agent (A2A) protocol. <https://a2aproject.github.io/A2A/latest/>, 2025.
- [60] Saaket Agashe, Jiuzhou Han, Shuyu Gan, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent S: An open agentic framework that uses computers like a human. *arXiv preprint arXiv:2410.08164*, 2024.
- [61] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, et al. OS-ATLAS: A foundation action model for generalist GUI agents. *arXiv preprint arXiv:2410.23218*, 2024.
- [62] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, et al. Windows agent arena: Evaluating multi-modal OS agents at scale. *arXiv preprint arXiv:2409.08264*, 2024.
- [63] Xueyu Hu, Tao Xiong, Biao Yi, Zishu Wei, Ruixuan Xiao, et al. OS agents: A survey on MLLM-based agents for general computing devices use. *arXiv preprint arXiv:2508.04482*, 2025.
- [64] Fei Tang, Haolei Xu, Hang Zhang, Siqi Chen, Xingyu Wu, et al. A survey on (M)LLM-based GUI agents. *arXiv preprint arXiv:2504.13865*, 2025.
- [65] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *NeurIPS*, 2024.
- [66] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *NeurIPS*, 2024.
- [67] Chengyou Jia, Minnan Luo, Zhuohang Dang, Qiushi Sun, et al. AgentStore: Scalable integration of heterogeneous agents as specialized generalist computer assistant. In *Findings of ACL*, pages 8908–8934, 2025.
- [68] Anthropic. Model context protocol (MCP) specification. <https://modelcontextprotocol.io/>, 2024.
- [69] Gao Chang, Eddie Lin, Chongyuan Yuan, Ran Cai, Bin Chen, Xinlin Xie, et al. Agent network protocol technical white paper. *arXiv preprint arXiv:2508.00007*, 2025.
- [70] Weijie Xu, Tianle Wang, Yuxuan Xia, Sheng Zhang, and Soung Chang Liew. Agent-OSI: A layered protocol stack toward a decentralized internet of agents. *arXiv preprint arXiv:2602.13795*, 2026.